



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

UNIVERSITY OF PADUA
DEPARTMENT OF INFORMATION ENGINEERING
MASTER'S DEGREE IN COMPUTER ENGINEERING

Git-Based Software Metrics for Evidence-Based Developer Reputation: Comparative Evaluation and Smart Contract Feasibility

MASTER CANDIDATE

Ali Bakhshayesh

Student ID 2071502

SUPERVISOR

Prof. Mauro Migliardi

ACADEMIC YEAR
2025/2026

GRADUATION DATE
10 SEPTEMBER 2026

Abstract

This thesis investigates whether Git-derived software metrics can support an evidence-based developer-reputation signal that is stable, interpretable, resistant to manipulation, and feasible within a smart-contract architecture. The empirical study constructs a reproducible dataset of ten open-source repositories, comprising 80 frozen release tags and 70 tag-chronological windows. It evaluates snapshot size, change magnitude, activity cadence, and ownership concentration through repository extraction, cleaning, bot-sensitivity analysis, duration normalization, and explicit comparison criteria.

The comparative evaluation identifies M4 developer churn ownership share as the strongest eligible baseline proof-of-concept metric under the declared framework. A post-hoc Git ancestry audit identifies 12 non-ancestral windows and retains 58 windows for sensitivity analysis. The behavioral stability ordering continues to support M4 as the engineering choice, while the limitation is retained explicitly rather than used to overwrite the original evidence. The selected metric is then frozen using deterministic integer arithmetic, versioned policies, and explicit evidence and ledger schemas.

The implementation adopts a hybrid trust boundary. Git retrieval, ancestry validation, identity normalization, bot classification, and metric extraction remain in a deterministic off-chain verifier, while Hyperledger Fabric chaincode performs bounded authorization, schema and arithmetic validation, immutable state commitment, indexing, history queries, and event emission. A standalone Git bundle and independent replay reproduce the selected fixture and its commitments byte-for-byte. All 34 local chaincode tests pass. Deployment to a two-organization Fabric test network subsequently passes all 17 authenticated Gateway checks and 47 independent network checks, with the accepted transaction independently confirmed as valid in block 6.

The result is a coherent and replayable proof of concept. It demonstrates that M4 can be committed as compact, auditable ledger evidence under an explicit policy and governance boundary. It does not claim that churn universally measures developer quality, that gaming resistance is empirically proven, or that the educational Fabric topology is production-ready.

Contents

List of Figures	xi
List of Tables	xiii
Abbreviations and Notation	xvii
1 Introduction	1
1.1 Introduction, Scope, and Research Roadmap	1
2 Foundations and Related Work	5
2.1 Candidate Metrics (Shortlist)	5
2.2 DESTINO Alignment and Evidence Boundary	6
2.2.1 Implications for This Thesis	7
3 Research Design and Methodology	9
3.1 Methodology: Comparison Criteria	9
3.1.1 Evaluation unit and normalization policy	9
3.1.2 C1 Stability	10
3.1.3 C2 Interpretability	10
3.1.4 C3 Resistance to Gaming	10
3.1.5 C4 Long-Term Suitability	11
3.1.6 Criterion-to-output mapping	11
3.1.7 Decision protocol (predefined, not executed in Phase I) . .	11
3.2 Extended Feasibility Rubric (Detailed)	12
3.2.1 Smart-Contract Feasibility Rubric	12
4 Dataset Construction and Descriptive Results	19
4.1 Phase I: Dataset Freeze and Version-Window Definition	19
4.1.1 Repository Freeze	19

CONTENTS

4.1.2	Tag Freeze and Consecutive Windows	20
4.2	Phase I: Metric Extraction Pipeline	22
4.2.1	Primary CSV outputs	22
4.2.2	Script-to-output mapping	24
4.2.3	Metric definitions (implementation-level)	24
4.2.4	Error handling and traceability	25
4.3	Phase I: Data Integrity Verification and Cleaning	26
4.3.1	Sanity checks	26
4.3.2	Canonical identity	26
4.3.3	Bot labeling	26
4.3.4	Known limitation: timestamp semantics	27
4.3.5	Issue-resolution ledger	28
4.4	Centralized Extraction Summary (Phase I Deliverables)	28
4.5	Detailed Descriptive Results for M1–M4	30
4.5.1	M1 Snapshot size evolution	30
4.5.2	M2 churn and M3 cadence intensity	31
4.5.3	Bot-sensitivity outputs	31
4.5.4	Duration-normalized analysis outputs	32
4.5.5	M3 developer activity-day profile	33
4.5.6	M4 ownership and concentration profile	33
4.6	Phase I Execution Ledger	34
4.6.1	Cleaning and validation outcomes	36
4.7	Reproducibility Protocol and Data Lineage	36
4.7.1	Artifact layout	36
4.7.2	Execution protocol	37
4.7.3	Lineage checkpoints	38
4.7.4	Automated validation evidence	38
4.8	Phase I Conclusion	39
5	Comparative Evaluation and Metric Selection	41
5.1	Phase II Step 1: C1 Stability Analysis	41
5.1.1	Analysis inputs and generated artifacts	41
5.1.2	Method	42
5.1.3	Window-level stability results	43
5.1.4	Developer-rank stability results	44
5.1.5	C1 interpretation for the decision process	45

5.2	Post-hoc Release-Window Ancestry Validation	46
5.2.1	Audit method and impact	46
5.2.2	Ancestry-only C1 sensitivity analysis	47
5.2.3	Reproducibility artifacts	47
5.3	Phase II Step 2: C2 Interpretability Analysis	48
5.3.1	Scoring protocol and artifacts	48
5.3.2	C2 results	49
5.3.3	C2 interpretation for the decision process	50
5.4	Phase II Step 3: C3 Gaming-Resistance Analysis	51
5.4.1	Scoring protocol and artifacts	51
5.4.2	Attack-surface summary	52
5.4.3	C3 results	53
5.4.4	C3 interpretation for the decision process	54
5.5	Phase II Step 4: C4 Long-Term Suitability Analysis	54
5.5.1	Scoring protocol and artifacts	55
5.5.2	Historical coverage	56
5.5.3	C4 results	56
5.5.4	C4 interpretation for the decision process	57
5.6	Phase II Step 5: Smart-Contract Feasibility Scoring	58
5.6.1	Scoring protocol and artifacts	58
5.6.2	Combined C1–C4 comparison	59
5.6.3	Feasibility results	60
5.6.4	Final selection inputs	61
5.7	Phase II Step 6: Final PoC Metric Selection	63
5.7.1	Selected metric	63
5.7.2	Selection artifacts	63
5.7.3	Why M4 is selected	64
5.7.4	Analysis policy	65
5.7.5	PoC record fields	66
5.7.6	Supporting signals retained	66
5.7.7	Decision boundary	67
6	Proof-of-Concept Design, Implementation, and Validation	69
6.1	DESTINO PoC Integration Blueprint	69
6.1.1	Design Objective and Boundary	69
6.1.2	Metric-Agnostic Record Shape	70

CONTENTS

6.1.3	Transaction and Replay Flow	70
6.1.4	How Phase I Outputs Support the Blueprint	71
6.2	Phase III: Deterministic M4 and Hybrid Fabric PoC Design	71
6.2.1	Versioned metric policy and arithmetic audit	72
6.2.2	Fabric platform profile	73
6.2.3	Off-chain/on-chain boundary	74
6.2.4	Evidence and ledger schemas	74
6.2.5	Trust model and residual threats	75
6.2.6	Baseline integration fixture	76
6.3	Phase III: Off-chain Verifier and Evidence Replay	76
6.3.1	Standalone Git evidence artifact	77
6.3.2	Direct verifier result	77
6.3.3	Fresh-bundle replay and independent traversal	78
6.3.4	Interpretation and off-chain boundary	79
6.4	Phase III: Chaincode Implementation and Local Validation	79
6.4.1	Validated state transition	80
6.4.2	Local test result	80
6.4.3	Interpretation and network boundary	81
6.5	Phase III: Fabric Network Deployment and Integration Validation	82
6.5.1	Frozen environment and deployment	82
6.5.2	Authenticated positive and negative paths	83
6.5.3	Committed transaction and independent validation	84
6.5.4	Interpretation and limits	85
7	Discussion and Conclusions	87
7.1	Threats to Validity	87
7.1.1	Construct validity	87
7.1.2	Internal validity	87
7.1.3	External validity	88
7.1.4	Conclusion validity	88
7.2	Discussion and Final Conclusion	89
7.2.1	Synthesis of the evidence	89
7.2.2	Answer to RQ1: stability and comparability	90
7.2.3	Answer to RQ2: interpretation and manipulation	91
7.2.4	Answer to RQ3: PoC selection and technical feasibility . .	92
7.2.5	Implications for a DESTINO-aligned reputation system . .	93

CONTENTS

7.2.6	Limitations and future work	93
7.2.7	Final conclusion	94
References		97

List of Figures

5.1	Distribution of per-project coefficient of variation for each C1 window-level signal.	43
5.2	Indexed C1 signal trajectories by project. Each line is scaled by that project's median value for the signal.	44
5.3	Adjacent-window developer-rank stability for M2, M3, and M4 developer-level signals.	45
5.4	C2 interpretability scores on a 0–10 scale. The design-extension metric is shown separately from implemented metrics.	50
5.5	C3 gaming-resistance scores on a 0–10 scale. Higher values indicate stronger resistance after safeguards.	53
5.6	C4 long-term suitability scores on a 0–10 scale. Higher values indicate stronger suitability for long histories.	57
5.7	Smart-contract feasibility scores on a 0–10 scale. Grey bars indicate metrics that are not eligible for the current PoC decision. . . .	61
5.8	Criterion comparison average, feasibility, and combined readiness scores used for the final selection gate. Only C1 is data-derived; C2–C4 are structured author assessments.	62

List of Tables

2.1	Candidate shortlist. All metrics are computed and compared before selecting a single PoC metric.	6
2.2	Alignment between DESTINO evidence requirements and current Phase I artifacts.	8
3.1	Operational mapping from methodology criteria to evidence sources.	11
3.2	Feasibility dimensions with detailed scoring rules (D1–D5).	14
3.3	Feasibility dimensions with detailed scoring rules (D6–D10).	15
3.4	Feasibility rubric scoring shape. Actual scores are reported in Section 5.6.	17
4.1	Centralized Phase I repository freeze.	20
4.2	Centralized version coverage by project.	21
4.3	Stable-tag rules for the centralized repositories.	21
4.4	Window-duration statistics.	22
4.5	Executable mapping from Phase I scripts to centralized outputs.	24
4.6	Project-level automation contribution from out/bot_summary_by_project_all.csv.	27
4.7	Centralized Phase I issue ledger and implemented validation actions.	28
4.8	Centralized project-level extraction summary from clean all-account and human-only outputs.	29
4.9	Centralized global extraction totals from all-account, human-only, and bot-only populations.	29
4.10	M1 snapshot-size statistics by project.	29
4.11	M1 code-line evolution across frozen tags. First and last values follow chronological tag order.	30

LIST OF TABLES

4.12	M2/M3 human-only intensity.	31
4.13	Automation impact summary. Percentages are relative to all-account project totals.	31
4.14	Short-window sensitivity flags. Windows below seven days are retained but marked as <code>sensitivity_only</code>	32
4.15	Highest human-only churn/day windows. Very short windows can dominate per-day rates, so they are not used as primary cadence evidence.	32
4.16	M3 human-only activity profile.	33
4.17	Mean human-only M4 concentration indicators.	33
4.18	Execution ledger for the centralized Phase I pipeline.	35
4.19	Centralized Phase I data lineage checkpoints used for reproducibility and auditing.	38
5.1	Phase II C1 stability artifacts. Paths are relative to the Phase II analysis directory.	42
5.2	C1 window-level volatility summary. CV is the coefficient of variation across project windows; “primary” excludes short windows marked <code>sensitivity_only</code>	43
5.3	C1 developer-rank stability summary across adjacent windows. Correlations use only developers appearing in both adjacent windows.	44
5.4	Post-hoc Git ancestry audit and quantitative impact.	46
5.5	Original and ancestry-only C1 window-stability results. High-volatility counts use only projects with at least two primary windows.	47
5.6	C2 interpretability rubric. Each dimension is scored from 0 to 2.	48
5.7	Phase II C2 interpretability artifacts. Paths are relative to the Phase II analysis directory.	49
5.8	C2 interpretability summary for the candidate metrics.	49
5.9	C3 gaming-resistance rubric. Each dimension is scored from 0 to 2, with higher scores meaning stronger resistance.	52
5.10	Phase II C3 gaming-resistance artifacts. Paths are relative to the Phase II analysis directory.	52
5.11	C3 gaming-resistance summary for the candidate metrics.	53

5.12	C4 long-term suitability rubric. Each dimension is scored from 0 to 2.	55
5.13	Phase II C4 long-term suitability artifacts. Paths are relative to the Phase II analysis directory.	55
5.14	C4 long-term suitability summary for the candidate metrics. . . .	56
5.15	Smart-contract feasibility scoring artifacts. Paths are relative to the Phase II analysis directory.	59
5.16	Combined C1–C4 decision inputs. C1 is derived from primary high-volatility project counts; C2–C4 use the scored summaries from the previous sections.	59
5.17	Smart-contract feasibility summary. Exclusion labels follow Section 3.2.1.	60
5.18	Final decision inputs before selecting the baseline PoC metric. . .	62
5.19	Final PoC metric-selection artifacts. Paths are relative to the Phase II analysis directory.	64
5.20	Selection decision inputs for the final PoC metric.	64
5.21	Selection-stage compact PoC record fields for the selected M4 metric.	66
6.1	Metric-agnostic record shape used during DESTINO PoC planning.	70
6.2	Deterministic M4 calculation audit.	72
6.3	Hybrid PoC computation and trust boundary.	74
6.4	Versioned evidence and ledger schemas for the baseline PoC. . . .	75
6.5	Direct Git extraction and M4 verifier result for urllib3_w03. . . .	78
6.6	Off-chain implementation commitments.	78
6.7	Local m4evidence chaincode validation result.	81
6.8	Chaincode implementation commitments.	81
6.9	Committed chaincode definition and client environment.	83
6.10	Authenticated Fabric Gateway integration checks.	84
7.1	Thesis-level answers and their evidence boundaries.	90

Abbreviations and Notation

Term	Meaning in this thesis
PoC	Proof-of-concept hybrid verifier and permissioned-ledger implementation.
DESTINO	Target evidence-based developer-reputation framework used to define the smart-contract evidence boundary.
Fabric	Hyperledger Fabric permissioned distributed-ledger platform used for the baseline PoC.
MSP	Membership Service Provider identity domain used by Fabric organizations.
SHA-256	Cryptographic hash used for policy, manifest, artifact, record, and pseudonymous identity commitments.
ppm	Parts per million; fixed-point scale used for the M4 ledger value.
LOC/SLOC	Total/source line counts used as snapshot-size indicators (M1).
Churn	Added plus deleted lines in a window (M2).
Cadence	Activity continuity over time (commits, active days, and gaps) (M3).
Ownership concentration	Distribution of developer contribution shares (M4).
Window	Adjacent tag-date pair ($T_k \rightarrow T_{k+1}$); PoC eligibility additionally requires Git ancestry.
All-account	Metric population including both human and automated Git author identities.
Human-only	Metric population excluding rows flagged as automated accounts via <code>is_bot</code> .
Pilot cohort	The first four repositories used to validate the workflow before expansion.
Extended cohort	Six additional repositories added to test the pipeline across more ecosystems.
Centralized Phase I dataset	Final 10-repository dataset used as the Phase I evidence base.



Introduction

1.1 INTRODUCTION, SCOPE, AND RESEARCH ROADMAP

This thesis studies repository-derived software indicators as evidence for developer reputation systems. The work constructs a reproducible multi-project dataset, extracts candidate metrics with transparent rules, compares their behavior and implementation suitability, and instantiates the selected signal in a hybrid permissioned-ledger design. Phase II selects M4 developer churn ownership share for the baseline proof-of-concept (PoC); Phase III freezes the deterministic calculation, evidence schemas, and Hyperledger Fabric trust boundary, implements and independently replays the off-chain verifier, locally validates the chaincode, and deploys the unchanged contract for authenticated two-organization Fabric integration testing.

POSITIONING RELATIVE TO PRIOR WORK AND DESTINO

Prior research provides a broad foundation for software measurement and assessment. Software-metric surveys and maintainability studies organize candidate measures and examine how measurable properties evolve, while open-source software-selection research treats metrics as decision inputs rather than complete judgments of software or developer quality [15, 9, 6, 13, 10]. Complementary work addresses the collection and exchange of metric data, third-party software assessment, and the evaluation of reputation indicators under explicit threat scenarios [7, 11, 8, 12].

1.1. INTRODUCTION, SCOPE, AND RESEARCH ROADMAP

The original DESTINO proposal places these concerns within a broader decentralized ecosystem for evidence-based software developer reputation. Its envisioned scope includes multiple software-quality and maintenance evidence types, automated measurement, signed submissions, immutable ledger records, evidence discovery, and developer profiles [1]. Implementing that complete ecosystem is outside the scope of this thesis.

This work instead addresses a narrower systems-assurance question within the DESTINO vision: once a Git-derived reputation indicator has been compared with alternatives and selected, how can its calculation, evidence packaging, submission, ledger commitment, and independent replay be made deterministic and auditable? Phases I and II provide the comparative evidence and select M4; Phase III operationalizes M4 through a deterministic off-chain verifier and a compact, validated Fabric record. Authorization in this PoC controls who may submit M4 evidence; M4 is not used to grant or deny developer permissions. The broad prior literature and the focused implementation are therefore complementary: the former establishes the measurement and assessment landscape, while this thesis evaluates and validates one end-to-end evidence path within it.

RESEARCH QUESTIONS

- **RQ1 (Stability):** Which candidate indicators are stable across versions and comparable across projects?
- **RQ2 (Interpretability and gaming):** Which indicators are understandable and robust against manipulation?
- **RQ3 (Smart-contract feasibility):** Which indicator is the strongest PoC candidate under implementation constraints?

DOCUMENT STRUCTURE

The report is organized as follows:

- Sections 2.1–6.1 define candidate metrics, comparison criteria, DESTINO alignment, feasibility scoring, and the metric-agnostic PoC integration boundary.

- Sections 4.1–4.4 document dataset curation, extraction, cleaning, and descriptive outputs for the centralized 10-repository Phase I dataset.
- Sections 4.7–4.8 provide the Phase I reproducibility protocol and conclusion.
- Sections 5.1–5.7 report the comparative evaluation, post-hoc ancestry sensitivity check, feasibility gate, and M4 selection.
- Sections 6.2–7.2 freeze the concrete hybrid PoC design, report the off-chain implementation and independent replay, validate the local chaincode implementation, and evaluate the live Fabric test-network boundary. Sections 7.1 and 7.2 then consolidate validity limits, answer the research questions, and state the final conclusion.

CONTRIBUTIONS

- A shortlist of candidate metrics and operational comparison criteria.
- A reproducible 10-repository Phase I dataset with frozen repositories, tags, and tag-chronological windows.
- Cleaned, validated metric datasets (developer-window and window totals) and ownership/concentration statistics.
- Explicit bot-sensitivity and short-window sensitivity outputs, so automation and release-cadence artifacts are visible rather than hidden.
- A post-hoc ancestry audit and non-overwriting C1 sensitivity analysis that make parallel release-line effects explicit.
- A transparent smart-contract feasibility rubric and frozen M4 selection with clearly separated empirical and structured-assessment evidence.
- A versioned DESTINO-aligned PoC design comprising deterministic integer encoding, evidence and ledger schemas, a two-organization Hyperledger Fabric topology, and an explicit replay/trust model.
- A standalone Git evidence bundle and deterministic off-chain verifier whose canonical manifest and ledger submission were reproduced byte-for-byte from a fresh repository and cross-checked through a second Git traversal.

1.1. INTRODUCTION, SCOPE, AND RESEARCH ROADMAP

- A schema-constrained JavaScript Fabric contract that recomputes M4 with exact BigInt arithmetic, rejects unauthorized or duplicate submissions, and passes 34 local state-transition and negative-case tests.
- A reproducible two-organization Fabric deployment in which all 17 authenticated Gateway checks and 47 independent network checks pass, including decoded two-peer endorsement, valid commit, peer-consistent reads, indexes, history, event delivery, and negative cases.



Foundations and Related Work

2.1 CANDIDATE METRICS (SHORTLIST)

This thesis compares a minimal set of candidate indicator families that are (i) computable from versioned artifacts and Git history, (ii) interpretable and comparable across heterogeneous projects, and (iii) compatible with hash-anchored evidence and compact on-chain storage. Source-code measurement and open-source component-selection research motivate treating repository metrics as decision inputs rather than complete quality judgments [15, 10]. Reputation-system research further motivates comparing multiple indicators under explicit threat scenarios [8, 12]. Research on metric-data infrastructures additionally treats collection, storage, and exchange as engineering concerns, while third-party assessment research shows that assessment objectives depend on the intended recipient and decision context [7, 11]. Finally, smart-contract studies establish platform-specific measurement constraints and caution against treating one complexity metric as a direct proxy for software quality or vulnerability [16, 14]. The comparative study evaluates all candidates; the PoC metric is selected only after evaluation.

2.2. DESTINO ALIGNMENT AND EVIDENCE BOUNDARY

Candidate	Usage in this thesis	Why chosen (one sentence)
M1 Snapshot size (LOC/SLOC)	Per-tag size baseline for normalization and context.	Simple, interpretable baseline; efficient to compute, store, and audit.
M2 Patch complexity proxy (churn)	Added/deleted lines per release window and developer.	Minimal proxy for change magnitude; aggregation-friendly and auditable.
M3 Activity cadence	Commits/activity-days per window; inactivity gaps.	Captures continuity over time without requiring platform-specific PR data.
M4 Ownership / concentration	Developer shares of churn/commits; top-1/top-3 and Gini concentration.	Separates core maintainers from occasional contributors; supports stability analysis.
M5 Maintenance responsiveness (design extension)	Documented schema/logic; not implemented in baseline PoC.	Included for completeness, but deferred to keep baseline PoC scope controlled.

Table 2.1: Candidate shortlist. All metrics are computed and compared before selecting a single PoC metric.

2.2 DESTINO ALIGNMENT AND EVIDENCE BOUNDARY

The original DESTINO proposal is the reference deployment context for this thesis. It describes a broad decentralized ecosystem in which software-quality and maintenance evidence contributes to developer reputation through automated measurement, signed submissions, immutable ledger records, evidence storage and discovery, and developer-oriented profiles [1].

This thesis intentionally implements a narrower part of that vision. It compares repository-derived candidate indicators, selects one signal for a baseline proof of concept, and evaluates whether its evidence can be calculated deterministically, submitted by an authorized party, committed with integrity

protections, and independently replayed. It does not implement qualified electronic signatures, maintenance-response evidence, aggregated reputation profiles, public search services, or production governance. Those broader DESTINO elements remain outside the claims evaluated here.

DESTINO is therefore not treated as a reason to move all computation on-chain. Instead, it defines an evidence boundary that every candidate metric must satisfy before a proof-of-concept metric can be selected. Within that boundary, authorization controls who may submit metric evidence; the selected metric does not itself grant or deny developer permissions.

The working DESTINO-aligned evidence model has four parts:

- a developer-facing submission containing project/archive metadata, software-type tags, storage-persistence metadata, and a cryptographic commitment to the submitted artifact;
- an off-chain measurement pipeline that retrieves the committed artifact, verifies integrity, and computes the selected metric;
- compact on-chain records containing the submission reference, metric value, and evidence hash;
- auditor-accessible replay: retrieve the same artifact, rerun the declared script and configuration, and compare the result with the committed metric record.

This boundary keeps expensive repository traversal, Git history analysis, and tabular validation off-chain, while the smart-contract layer stores only bounded metadata, commitments, and metric outputs. This matches common blockchain engineering constraints: large artifacts and unbounded loops are poor on-chain candidates, while hash commitments and compact records are auditable and storage-bounded [2, 16].

2.2.1 IMPLICATIONS FOR THIS THESIS

Phase I is therefore not solely a data-extraction phase. It prepares the evidence artifacts subsequently used by the DESTINO-aligned PoC. The alignment is summarized in Table 2.2.

2.2. DESTINO ALIGNMENT AND EVIDENCE BOUNDARY

DESTINO need	Phase I artifact or rule	Why it matters
Stable artifact identity	<code>data/dataset_freeze_all.csv</code> , <code>data/versions_all.csv</code> , frozen commit hashes	Metric evidence can be tied to explicit projects, tags, and repository states.
Replayable measurement interval	<code>data/windows_all.csv</code>	Each metric value has a deterministic release-window boundary.
Compact metric evidence	<code>out/loc_per_tag_all.csv</code> , clean metric tables, normalized tables, human-only M3/M4 tables	The contract can store selected values without storing raw repositories or diffs.
Audit and validation evidence	<code>logs/*.log</code> , cleaning checks, duration flags, bot-sensitivity summaries	A verifier can inspect how raw extraction became analysis-ready evidence.
Smart-contract selection discipline	Criteria C1–C4 plus the feasibility rubric	The selected PoC metric must be empirically useful and feasible under bounded contract execution.

Table 2.2: Alignment between DESTINO evidence requirements and current Phase I artifacts.

Two Phase I design decisions are especially important for DESTINO alignment. First, duration-normalized outputs and `sensitivity_only` flags prevent release cadence artifacts from being silently recorded as comparable metric evidence. Second, human-only M3/M4 outputs prevent automated accounts from being interpreted as human maintainer behavior. The all-account tables remain useful as an audit trail, but the primary developer-behavior evidence is explicitly human-only.

No PoC metric is selected within Phase I itself. At that phase boundary, DESTINO alignment means the pipeline produces evidence that can subsequently be committed, audited, and scored; Phase II performs the comparative and feasibility gate.



Research Design and Methodology

3.1 METHODOLOGY: COMPARISON CRITERIA

Candidates are compared using four criteria: stability, interpretability, resistance to gaming, and long-term suitability. The comparative phase (Phase II) applies these criteria uniformly to M1–M5 before any PoC metric decision.

The scoring labels are local to each analysis step. C2, C3, and C4 each use five criterion-specific dimensions: C2_D1–C2_D5, C3_D1–C3_D5, and C4_D1–C4_D5. These are separate from the smart-contract feasibility rubric, which later uses ten dimensions, D1–D10, in Section 3.2.1.

3.1.1 EVALUATION UNIT AND NORMALIZATION POLICY

The primary analysis unit is (project_id, window_id, developer) for developer-level metrics and (project_id, window_id) for aggregate metrics. Because window durations and project sizes vary, the pipeline produces both raw and normalized values:

- per-day normalization for cadence-sensitive measures;
- per-KLOC normalization where size effects can dominate;
- within-project shares (ownership) to control for absolute scale.

Windows shorter than seven days are retained for auditability but flagged as sensitivity_only; Phase II reports both full-dataset results and robust results

3.1. METHODOLOGY: COMPARISON CRITERIA

excluding those very short windows when duration-sensitive metrics are evaluated.

Automation is handled as an explicit population rule. Rows flagged with `is_bot` remain in the clean all-account tables for auditability and project-activity totals. However, developer-behavior metrics use human-only outputs as the primary population. In the centralized Phase I dataset, M3 cadence and M4 ownership/concentration are evaluated from `out/metrics_window_dev_human_all.csv` and `out/ownership_summary_human_all.csv`. Bot summaries are reported as sensitivity evidence rather than silently mixed into maintainer-behavior claims.

3.1.2 C1 STABILITY

Stability is evaluated across consecutive version windows. The analysis measures volatility (e.g., dispersion across windows) and rank stability (e.g., correlation of developer rankings between adjacent windows). Cross-project comparison is performed after simple normalizations (e.g., per day or per KLOC) to reduce artifacts from project size and release cadence. The output set comprises dispersion statistics, adjacent-window rank correlations, and sensitivity checks on timestamp semantics.

3.1.3 C2 INTERPRETABILITY

Interpretability is assessed via semantic clarity (plain meaning), unit clarity (counts/shares/time), and normalization clarity. Metrics that require complex tooling or ambiguous definitions receive lower interpretability.

3.1.4 C3 RESISTANCE TO GAMING

The analysis examines attack surfaces and manipulation cost for each metric. Since individual metrics can be misleading, the interpretation avoids strong causal claims and treats gaming risks comparatively (e.g., commit-splitting inflates frequency but becomes detectable when churn remains low). The threat model follows reputation-system evaluation research that compares metric behavior under hostile-agent scenarios and other explicit attacks [8, 12]. Strong causal interpretations are avoided because code-complexity measures can be useful as complementary indicators without any single measure having a strong association with vulnerability [14].

3.1.5 C4 LONG-TERM SUITABILITY

The assessment considers whether each metric remains informative over long histories and across lifecycle phases, and whether it supports incremental updates without full recomputation. This lifecycle emphasis follows maintainability research that treats changeability, evolution, and paradigm-sensitive measurement as distinct software-quality concerns [9, 6, 13].

3.1.6 CRITERION-TO-OUTPUT MAPPING

Criterion	Phase II evidence	Primary data sources
C1 Stability	Window-level volatility, adjacent-window rank correlation, robustness to normalization choices.	out/metrics_window_dev_human_normalized_all.csv, out/metrics_window_totals_human_normalized_all.csv, out/window_duration_flags_all.csv
C2 Interpretability	Structured rubric (semantic clarity, unit clarity, normalization clarity, tooling transparency).	Metric definitions plus extracted fields in out/artifacts
C3 Gaming resistance	Attack-surface matrix and manipulation detectability checks under multi-metric reading.	out/metrics_window_dev_human_all.csv, out/ownership_dev_shares_human_all.csv, out/bot_summary_by_project_all.csv
C4 Long-term suitability	Informativeness over time, lifecycle robustness, and incremental update feasibility.	out/loc_per_tag_all.csv, data/windows_all.csv, feasibility rubric dimensions

Table 3.1: Operational mapping from methodology criteria to evidence sources.

3.1.7 DECISION PROTOCOL (PREDEFINED, NOT EXECUTED IN PHASE I)

The decision protocol is fixed before Phase II analysis:

1. compute comparative evidence for all candidates under C1–C4;
2. score smart-contract feasibility using the rubric in Section 3.2.1;
3. eliminate any candidate that violates feasibility exclusion criteria;
4. select one PoC metric with the best overall trade-off between comparative behavior and feasibility.

3.2. EXTENDED FEASIBILITY RUBRIC (DETAILED)

No step of this decision protocol is executed in Phase I.

3.2 EXTENDED FEASIBILITY RUBRIC (DETAILED)

3.2.1 SMART-CONTRACT FEASIBILITY RUBRIC

WHY FEASIBILITY MUST BE A FIRST-CLASS CRITERION

This thesis evaluates multiple candidate metrics for evidence-based developer reputation. However, the project explicitly requires that feasibility as a smart contract is considered as a selection criterion and that a proof-of-concept (PoC) is implemented for one chosen metric. The project scope further constrains the design through a conceptual evidence workflow where developers submit a signed form containing a link and a hash of an archive, the artifact and metric are verified, compact evidence is written to the ledger, and a sibling search/index system can observe each new data point. Because Fabric chaincode cannot retrieve an external archive or traverse Git, the implemented interpretation assigns retrieval and metric computation to a deterministic off-chain verifier and reserves bounded authorization, validation, commitment, state, and event operations for chaincode. Therefore, feasibility here is not a generic “blockchain compatibility” notion, but the practical ability to implement (or safely approximate) the required pipeline under smart-contract constraints while preserving auditability and integrity.

In the blockchain literature, feasibility is shaped by system constraints that include (i) performance limits and throughput constraints, (ii) storage growth and distribution overhead, and (iii) challenges in connecting on-chain logic to external services and data (oracle-style issues). These constraints make heavy computation and large state growth poor candidates for on-chain implementation, often motivating hybrid architectures in which bulk data remains off-chain and the chain stores commitments and compact metadata [2]. Smart-contract ecosystems add further constraints: execution is costed (gas), code and execution patterns are shaped by platform limits, and determinism is required. Empirical work on Solidity metrics shows that many metric distributions are truncated and exist in restricted ranges due to platform constraints, which supports treating feasibility as a distinct and non-trivial selection criterion rather than assuming traditional software metrics transfer unchanged [16].

DEFINITION: SMART-CONTRACT FEASIBILITY

This thesis defines the **smart-contract feasibility** of a candidate metric as:

The ability to compute, verify, and store a metric (or its verified output) within the defined evidence workflow with bounded on-chain computation and storage, minimal reliance on unverifiable external inputs, strong auditability (re-checking from archived evidence), and explicit binding to developer identity/provenance.

This definition is intentionally *architecture-aware*. A candidate can be feasible in one of two ways:

- **On-chain computable feasibility:** the metric can be computed directly in the contract with small, bounded logic.
- **Hybrid feasibility (recommended for the PoC):** the metric is computed off-chain from an artifact whose hash and link are verified, and the contract stores only compact metric values plus hash commitments to evidence/inputs to enable auditability.

The rubric below evaluates feasibility in a way that allows both patterns, while explicitly penalizing designs that require unverifiable oracles, frequent high-throughput writes, or large on-chain state.

RUBRIC STRUCTURE AND SCORING

Each metric receives a score on 10 dimensions, each scored in $\{0, 1, 2\}$ (0 = poor feasibility, 2 = strong feasibility). The rubric is used **only after** the multi-metric comparative analysis, to select the single PoC metric.

Rubric application procedure.

1. For each candidate metric M_i , record a one-paragraph implementation account specifying the inputs, computation, stored values, update frequency, and attesting party.
2. Score each rubric dimension using the guidance below.
3. Compute a total feasibility score and also record *exclusion conditions* (dimensions where score = 0 implies unacceptable risk for the PoC).

3.2. EXTENDED FEASIBILITY RUBRIC (DETAILED)

4. Select the PoC metric as the best trade-off between (i) comparative performance under C1–C4 and (ii) feasibility score, with tie-breaking favoring determinism, auditability, and bounded cost.

FEASIBILITY DIMENSIONS (DETAILED)

Dimension	0–2	Scoring rule (detailed)
D1. Determinism of computation	0–2	2: metric computation is deterministic given the archived artifact and explicit parameters (window definition, tool version). 1: deterministic only under additional assumptions (toolchain pinned; minor nondeterminism mitigated). 0: depends on nondeterministic factors or ambiguous extraction (e.g., floating-point randomness, unstable parsing).
D2. Bounded on-chain computation	0–2	2: on-chain logic is constant-time or small bounded checks (signature format check, hash equality, record writes). 1: moderate bounded computation per update (limited loops over small arrays). 0: requires heavy analysis on-chain (AST parsing, full-history scanning) or unbounded loops.
D3. Bounded on-chain storage growth	0–2	2: store only compact values + commitments (hashes) + minimal metadata. 1: store moderate aggregates (rolling sums, bounded windows). 0: requires storing large histories/artifacts or per-commit detailed logs on-chain.
D4. Update frequency feasibility	0–2	2: metric can be updated per release or per time window (low write rate). 1: requires medium-frequency updates (e.g., per PR/issue) but can be batched. 0: requires high-frequency updates (per commit/event stream) and becomes throughput-limited.
D5. External data/oracle dependence	0–2	2: inputs are self-contained in the archived artifact + signed form, verifiable by hash commitments. 1: uses off-chain data but can be posted as signed attestations with clear trust model. 0: requires unverifiable external services or subjective judgments (strong oracle problem).

Table 3.2: Feasibility dimensions with detailed scoring rules (D1–D5).

Dimension	0–2	Scoring rule (detailed)
D6. Auditability and re-checking	0–2	2: third parties can recompute metric from archive retrieved via link and verified by on-chain hash; measurement report is committed. 1: reproducible with effort (tooling dependencies; partial evidence). 0: cannot be reliably reproduced or verified from public evidence.
D7. Identity binding and provenance	0–2	2: metric evidence is bound to developer identity via signed submissions (and the same identity used consistently across submissions). 1: partial binding (mapping from commit authors; weaker identity guarantees). 0: attribution cannot be trusted or can be trivially spoofed.
D8. Integrity of artifact linkage	0–2	2: strong commitment: archive hash stored on-chain and verified against retrieved archive; versioning explicit. 1: link is stable but hash commitment incomplete or relies on weak assumptions. 0: artifact cannot be pinned; link rot or mutable artifact breaks verification.
D9. Metric-value representation (range + precision)	0–2	2: metric stored as small integers or fixed-point with clear bounds; low risk of overflow/precision issues. 1: requires careful normalization/scale but feasible (e.g., store numerator/denominator separately). 0: requires floating point or complex statistics not representable safely on-chain.
D10. Adversarial resilience in the configured ledger context	0–2	2: metric is hard/expensive to inflate without leaving detectable traces, especially with hash-anchored evidence. 1: partially gameable but mitigations exist (normalization, batching, cross-check with other metrics in study). 0: trivially gameable with low-cost manipulations (e.g., verbosity inflation) and weak detectability.

Table 3.3: Feasibility dimensions with detailed scoring rules (D6–D10).

EVIDENCE CHECKLIST FOR EACH DIMENSION

To keep feasibility rigorous and evidence-based, each dimension must be supported by explicit evidence:

- **D1 Determinism:** exact inputs (archive hash, link, and version identifier), the window definition, and, for off-chain computation, the pinned tool version and configuration; an explanation of how the PoC ensures that the same artifact yields the same output.
- **D2/D3 Cost bounds:** the contract operations required per data point (comparisons and storage writes), together with a boundedness justification.
- **D4 Update frequency:** the update cadence (per release or time window) and a justification for avoiding per-commit updates.
- **D5 Oracle dependence:** any off-chain sources beyond the archive and, where applicable, whether they are hash-anchored or signed attestations.
- **D6 Auditability:** the *re-check protocol*: archive retrieval, hash verification, metric recomputation, and comparison with the ledger value.

3.2. EXTENDED FEASIBILITY RUBRIC (DETAILED)

- **D7/D8 Provenance and linkage:** an explanation of how the signed form and hash commitment prevent artifact substitution.
- **D9 Representation:** the encoding (integer, fixed-point, or separate numerator and denominator), overflow handling, and normalization.
- **D10 Adversarial model:** low-cost manipulations and whether evidence-consistency checks can detect them.

FEASIBILITY EXCLUSION CONDITIONS

Even if a metric is strong in the comparison, it is a weak PoC candidate if it triggers exclusion conditions:

- **Exclusion Condition A:** score 0 on D1 (non-deterministic), D6 (non-auditable), or D8 (artifact cannot be pinned).
- **Exclusion Condition B:** requires high-frequency updates ($D4=0$) that exceed realistic ledger throughput.
- **Exclusion Condition C:** depends on unverifiable external facts ($D5=0$) without a signed-attestation model.

MAPPING TO THE PROJECT EVIDENCE WORKFLOW

The conceptual project workflow comprises signature checking, archive retrieval, hash verification, metric measurement, ledger recording, and indexing notification. The rubric operationalizes this in dimensions D1, D6, D7, and D8 (determinism, auditability, identity binding, artifact linkage), and constrains practical implementation via D2–D5 and D9 (bounded compute/storage, update frequency, oracle dependence, representation). Section 6.2.3 records the implemented split: archive and Git operations are off-chain, whereas chaincode performs bounded validation, commitment, state, and event operations.

APPLICATION TO THE FIVE THESIS CANDIDATES

Table 3.4 records the scoring shape used for each candidate. The actual scored application is reported after C1–C4 in Section 5.6, where each dimension receives a value and rationale.

Metric	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
M1 Snapshot size (LOC/SLOC)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M2 Patch complexity (churn)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M3 Activity cadence	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M4 Ownership/concentration	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M5 Maintenance responsiveness (design ext.)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Table 3.4: Feasibility rubric scoring shape. Actual scores are reported in Section 5.6.

4

Dataset Construction and Descriptive Results

4.1 PHASE I: DATASET FREEZE AND VERSION-WINDOW DEFINITION

This section documents the completed Phase I freeze workflow for the centralized dataset: repository metadata freeze, stable-tag freeze, and consecutive-window construction. The first four repositories were used as a pilot to validate the workflow. The final Phase I dataset keeps those four repositories and adds six more projects from different ecosystems. The cohort labels are retained only as provenance; the analysis dataset is centralized.

4.1.1 REPOSITORY FREEZE

The centralized Phase I dataset contains ten mature GitHub-hosted repositories. Storage persistence is recorded as High (GitHub) for every project. Software-type tags are kept short and descriptive so that later analysis can group projects without losing the identity of each repository.

4.1. PHASE I: DATASET FREEZE AND VERSION-WINDOW DEFINITION

Cohort	Project	Repository URL	Software-type tags
Pilot	flask	https://github.com/pallets/flask	web-framework, library
Pilot	requests	https://github.com/psf/requests	http-client, library
Pilot	urllib3	https://github.com/urllib3/urllib3	http-client, library
Pilot	click	https://github.com/pallets/click	cli-toolkit, library
Extended	brew	https://github.com/Homebrew/brew	package-manager, cli-tool, ecosystem-infrastructure
Extended	curl	https://github.com/curl/curl	network-transfer-tool, cli-tool, library
Extended	gin	https://github.com/gin-gonic/gin	web-framework, http-framework, library
Extended	junit5	https://github.com/junit-team/junit5	testing-framework, library
Extended	prettier	https://github.com/prettier/prettier	code-formatter, cli-tool, developer-tool
Extended	tokio	https://github.com/tokio-rs/tokio	async-runtime, concurrency-library, library

Table 4.1: Centralized Phase I repository freeze.

4.1.2 TAG FREEZE AND CONSECUTIVE WINDOWS

For each project, eight stable release tags are frozen. Let $\{T_1, \dots, T_8\}$ be the frozen tags sorted chronologically by tag date from oldest to newest. Seven tag-chronological windows are then defined as $W_k = (T_k \rightarrow T_{k+1})$ for $k \in \{1, \dots, 7\}$. Each frozen tag is bound to a specific commit hash, including dereferencing annotated tags to their target commits. This gives each metric value a replayable repository state and deterministic Git revision expression. It does not by itself guarantee that every adjacent tag pair belongs to one ancestor-to-descendant release line; that condition is evaluated separately in Section 5.2.

The final freeze is captured in `data/dataset_freeze_all.csv`, `data/versions_all.csv`, and `data/windows_all.csv`. The earlier pilot-only CSVs were consolidated into these centralized files and removed from the active evidence layer to avoid maintaining parallel copies of the same dataset.

Centralized Phase I totals are:

- 10 repositories;
- 80 frozen tags (10 projects $\times$ 8 tags);
- 70 consecutive windows (10 projects $\times$ 7 windows);

- tag-date coverage from 2022-06-06 to 2026-04-29.

Project	Tags	Windows	First tag	Last tag
brew	8	7	5.1.1	5.1.8
click	8	7	8.1.6	8.3.1
curl	8	7	curl-8_14_0	curl-8_20_0
flask	8	7	3.0.0	3.1.3
gin	8	7	v1.8.1	v1.12.0
junit5	8	7	r6.0.0	r5.14.4
prettier	8	7	3.7.1	3.8.3
requests	8	7	v2.30.0	v2.32.5
tokio	8	7	tokio-1.47.3	tokio-1.52.1
urllib3	8	7	2.2.3	2.6.3

Table 4.2: Centralized version coverage by project.

Project	Stable tag rule	Reason for project-specific rule
brew	<code>~\d+\.\d+\.\d+\$</code>	Plain semantic-version release tags.
click	<code>~\d+\.\d+\.\d+\$</code>	Plain semantic-version release tags.
curl	<code>~curl-\d+_\d+_\d+\$</code>	Stable curl releases use a <code>curl-</code> prefix and underscores; release candidates are excluded.
flask	<code>~\d+\.\d+\.\d+\$</code>	Plain semantic-version release tags.
gin	<code>~v\d+\.\d+\.\d+\$</code>	Stable Go module tags use a <code>v</code> prefix.
junit5	<code>~r\d+\.\d+\.\d+\$</code>	Stable JUnit tags use an <code>r</code> prefix; milestone and release-candidate tags are excluded.
prettier	<code>~\d+\.\d+\.\d+\$</code>	Stable releases use plain semantic versions; alpha and non-version tags are excluded.
requests	<code>~v\d+\.\d+\.\d+\$</code>	Stable release tags use a <code>v</code> prefix.
tokio	<code>~tokio-\d+\.\d+\.\d+\$</code>	The repository contains crate-specific tags; only main Tokio runtime releases are included.
urllib3	<code>~\d+\.\d+\.\d+\$</code>	Plain semantic-version release tags.

Table 4.3: Stable-tag rules for the centralized repositories.

4.2. PHASE I: METRIC EXTRACTION PIPELINE

Project	Min window days	Max window days	Mean window days
brew	0.83	8.96	4.99
click	10.04	489.99	121.58
curl	7.00	63.00	48.00
flask	16.05	219.96	124.66
gin	60.82	378.26	194.70
junit5	0.03	69.91	29.73
prettier	0.87	78.36	19.79
requests	0.26	376.04	119.74
tokio	0.58	58.99	14.87
urllib3	3.00	170.05	68.89

Table 4.4: Window-duration statistics for the centralized release windows.

Two details need to be kept in view during Phase II. First, some repositories release several stable tags very close together, especially `junit5`, `brew`, `prettier`, and `tokio`. Second, JUnit has overlapping stable lines in the frozen interval. This is preserved in the original tag-date freeze for reproducibility, but it cannot be handled by short-window flags alone. The post-hoc ancestry audit therefore identifies divergent pairs explicitly, excludes them from the ancestry-only C1 sensitivity analysis, and makes them ineligible for the baseline PoC.

4.2 PHASE I: METRIC EXTRACTION PIPELINE

Candidate metrics M1–M4 are computed off-chain from Git history and frozen tag windows. The scripts are deterministic command-line programs: they consume declared CSV inputs and write CSV outputs without manual editing. The centralized execution consumes `data/dataset_freeze_all.csv`, `data/versions_all.csv`, and `data/windows_all.csv`. Pilot-specific files remain in the workspace for traceability, but the suffixed `_all` artifacts are the final Phase I evidence base.

4.2.1 PRIMARY CSV OUTPUTS

- `out/loc_per_tag_all.csv`: per (project, tag) snapshot size with total, blank, comment, and code lines.
- `out/metrics_window_dev_all.csv`: per (project, window, developer) commits, added/deleted lines, churn, binary-file count, activity days, and first/last committer timestamps.

- `out/metrics_window_totals_all.csv`: per (project, window) totals and unique-author counts.
- `out/metrics_window_dev_clean_all.csv` and `out/metrics_window_totals_clean_all.csv`: validated clean datasets used as the all-account audit baseline.
- `out/ownership_dev_shares_clean_all.csv` and `out/ownership_summary_clean_all.csv`: all-account ownership and concentration computed strictly from clean data.
- `out/metrics_window_dev_human_all.csv`, `out/metrics_window_totals_human_all.csv`, and `out/ownership_summary_human_all.csv`: human-only M3/M4 analysis population.
- `out/metrics_window_dev_human_normalized_all.csv` and `out/metrics_window_totals_human_normalized_all.csv`: duration-normalized human-only metrics.
- `out/window_duration_flags_all.csv` and `out/window_duration_sensitivity_summary_all.csv`: short-window flags and sensitivity thresholds.
- `out/bot_summary_by_project_all.csv`, `out/bot_summary_by_window_all.csv`, and `out/bot_identity_summary_all.csv`: automation impact summaries.

4.2. PHASE I: METRIC EXTRACTION PIPELINE

4.2.2 SCRIPT-TO-OUTPUT MAPPING

Script	Primary role in the centralized run
<code>scripts/extract_window_metrics.py</code>	Extracts M2/M3 base data from Git revision ranges and flags known automated identities.
<code>scripts/compute_loc_per_tag.py</code>	Legacy Python-only M1 extractor retained for audit continuity with the pilot run.
<code>scripts/compute_loc_per_tag_multilang.py</code>	Computes multi-language M1 snapshots for the expanded dataset using tracked source files.
<code>scripts/clean_round1_metrics.py</code>	Canonicalizes developer-window rows by (<code>project_id</code> , <code>window_id</code> , <code>author_email</code>) and recomputes clean totals.
<code>scripts/recompute_ownership_from_clean.py</code>	Computes clean ownership shares, top-1/top-3 concentration, and Gini values.
<code>scripts/compute_bot_sensitivity_outputs.py</code>	Exports human-only tables and bot-sensitivity summaries while keeping all-account outputs for audit.
<code>scripts/compute_duration_normalized_metrics.py</code>	Computes per-day and per-KLOC-day fields and marks short windows as <code>sensitivity_only</code> .

Table 4.5: Executable mapping from Phase I scripts to centralized outputs.

4.2.3 METRIC DEFINITIONS (IMPLEMENTATION-LEVEL)

M1 (LOC/SLOC). For each frozen tag commit, source files are counted and classified into blank, comment, and code lines. The centralized run uses a multi-language policy documented in `data/loc_language_policy_all.csv`; it counts tracked source files for the main languages of the ten repositories and excludes build, cache, dependency, and version-control directories. This keeps M1 comparable as a source-size indicator while avoiding generated or vendored material where possible.

M2 (Churn). For each window ($from_tag \rightarrow to_tag$), Git `numstat` output is used to sum lines added and deleted per developer. Churn is defined as added + deleted. Binary file changes are not assigned line counts, but they are retained through `binary_file_count`.

M3 (Cadence). Cadence is represented by commit counts, activity days, and first/last committer timestamps per developer-window. Human-only cadence outputs are the primary developer-behavior population. All-account cadence outputs are retained as an audit baseline because they describe total repository activity, including automated updates.

M4 (Ownership and concentration). For each project-window, developer shares are computed as:

$$share_{d,w}^{(churn)} = \frac{churn_{d,w}}{\sum_{d'} churn_{d',w}}, \quad share_{d,w}^{(commits)} = \frac{commits_{d,w}}{\sum_{d'} commits_{d',w}}.$$

The pipeline derives top-1/top-3 shares and Gini concentration values. Because M4 is intended to describe human maintainership concentration, the primary M4 outputs are the human-only tables. All-account M4 remains useful for audit and sensitivity analysis.

Duration-normalized derived metrics. Release windows vary from less than one hour to more than one year. The pipeline therefore derives `commits_per_day`, `churn_per_day`, and `churn_per_kloc_day`. Windows shorter than seven days are retained for traceability but marked as `sensitivity_only`. Phase II should not let these windows drive primary cadence conclusions.

Bot-sensitivity derived metrics. Automated accounts are explicitly labeled through `is_bot`. The final centralized dataset flags six automation identities: `dependabot[bot]`, `renovate[bot]`, `pre-commit-ci[bot]`, `pre-commit-ci-lite[bot]`, `autofix-ci[bot]`, and `BrewTestBot`. The bot-sensitivity outputs quantify their contribution and export a parallel human-only population for M3 and M4.

4.2.4 ERROR HANDLING AND TRACEABILITY

The extraction pipeline is fault-tolerant at repository-window granularity:

- Git command failures are appended to the corresponding `logs/*.log` file and processing continues where possible;
- cleaning, ownership, duration-normalization, and bot-sensitivity scripts write structured logs with row counts and validation outcomes;
- all derived tables are built from declared input files and script rules, not from manual spreadsheet edits.

This gives the report a reproducible audit trail: a reader can follow the dataset from frozen repositories and commits to clean, human-only, normalized metric tables.

4.3 PHASE I: DATA INTEGRITY VERIFICATION AND CLEANING

4.3.1 SANITY CHECKS

The final Phase I dataset is checked at three levels:

- window integrity: each of the ten projects has seven consecutive windows from eight frozen tags;
- row arithmetic: for every developer-window row, $\text{churn} = \text{added} + \text{deleted}$;
- aggregation consistency: window totals equal the sums of the corresponding developer rows.

All 70 centralized windows pass these checks. No clean window has zero commits or zero churn. The all-account clean dataset contains 1,118 developer-window rows, and the centralized window-total table contains 70 rows.

4.3.2 CANONICAL IDENTITY

Developer identity is canonicalized during extraction using the key:

`(project_id, window_id, author_email)`

For each key, commits are aggregated by email address and a canonical `author_name` is selected using the most frequent non-unknown display name. The cleaning pass found no duplicate key groups in either the extended cohort or the centralized dataset:

- extended dataset: 845 input rows, 845 clean rows, 0 duplicate groups merged;
- centralized dataset: 1,118 input rows, 1,118 clean rows, 0 duplicate groups merged.

4.3.3 BOT LABELING

Automated accounts are retained in the clean all-account dataset and marked with `is_bot`. This makes the automation policy visible and allows two populations to be reported:

- all-account outputs, used as the audit baseline for total repository activity;
- human-only outputs, used as the primary population for M3 cadence and M4 ownership/concentration.

The centralized clean dataset contains 55 bot developer-window rows and 1,063 human developer-window rows. Bot accounts contribute 999 of 9,560 commits (10.45%) but only 9,294 of 2,066,059 churn lines (0.45%). This confirms that automation affects commit-frequency and ownership/cadence interpretation more than aggregate churn magnitude.

Project	Bot rows	Bot commits	Bot commit %	Bot churn %
brew	10	67	17.72	9.10
click	8	55	13.03	0.94
curl	10	127	3.78	0.33
flask	2	24	13.33	1.70
gin	5	100	25.32	5.15
junit5	7	556	14.10	0.12
prettier	4	5	6.17	0.94
requests	3	27	17.76	4.94
tokio	1	1	0.20	0.00
urllib3	5	37	25.00	2.19

Table 4.6: Project-level automation contribution from `out/bot_summary_by_project_all.csv`.

4.3.4 KNOWN LIMITATION: TIMESTAMP SEMANTICS

Some commits included in a tag-to-tag revision range may have commit-ter timestamps outside the tag-date interval. This can happen when work was committed on a branch before it became reachable from a later tagged release. Churn, commit counts, and ownership shares are computed from Git revision ranges and are not affected by this timestamp drift. Cadence features based on first/last commit timestamps are more sensitive, so Phase II uses human-only commit counts and duration-normalized rates as the primary cadence evidence and treats timestamp-derived active-day fields as sensitivity evidence.

4.4. CENTRALIZED EXTRACTION SUMMARY (PHASE I DELIVERABLES)

4.3.5 ISSUE-RESOLUTION LEDGER

Issue	Observed count	Resolution in centralized Phase I dataset
Display-name variation for the same email/window	0 duplicate groups after cleaning	Canonical aggregation by email key is performed during extraction and checked during cleaning.
Incorrect unique-author counting in totals	0 mismatches	Window totals compute unique authors from canonical <code>author_email</code> values.
Bot contamination in M3/M4	55 bot rows	Bots are retained for audit, human-only M3/M4 outputs are exported, and bot contribution is summarized by project, window, and identity.
Very short release windows	22 windows below 7 days	These windows are retained but marked <code>sensitivity_only</code> ; robust cadence analysis excludes them.
Timestamp drift between commit dates and tag dates	Documented limitation	Revision-range metrics remain valid; timestamp-derived cadence fields require sensitivity checks.

Table 4.7: Centralized Phase I issue ledger and implemented validation actions.

4.4 CENTRALIZED EXTRACTION SUMMARY (PHASE I DELIVERABLES)

The centralized Phase I extraction spans 70 windows and produces 1,118 clean all-account developer-window rows after identity canonicalization. Aggregate all-account totals are 9,560 commits and 2,066,059 churn lines. The primary human-only population for M3/M4 contains 1,063 developer-window rows, 8,561 commits, and 2,056,765 churn lines.

Project	Windows	All commits	All churn	Human commits	Human churn
brew	7	378	43,524	311	39,565
click	7	422	29,176	367	28,901
curl	7	3,356	421,663	3,229	420,253
flask	7	180	13,159	156	12,935
gin	7	395	22,079	295	20,943
junit5	7	3,942	1,456,803	3,386	1,455,032
prettier	7	81	8,194	76	8,117
requests	7	152	3,277	125	3,115
tokio	7	506	55,498	505	55,496
urllib3	7	148	12,686	111	12,408

Table 4.8: Centralized project-level extraction summary from clean all-account and human-only outputs.

Scope	Windows	Developer rows	Total commits	Total churn
All accounts	70	1,118	9,560	2,066,059
Human-only M3/M4 population	70	1,063	8,561	2,056,765
Automated accounts only	42	55	999	9,294

Table 4.9: Centralized global extraction totals from all-account, human-only, and bot-only populations.

For the automated-account row, the window count reports windows with at least one bot row.

Project	First code lines	Last code lines	Delta	Mean code lines
brew	213,073	220,989	7,916	217,077
click	13,454	16,504	3,050	14,691
curl	224,573	225,738	1,165	222,721
flask	12,586	12,977	391	12,816
gin	13,510	17,974	4,464	14,730
junit5	132,705	132,689	-16	132,821
prettier	112,668	113,217	549	113,035
requests	7,964	8,412	448	8,257
tokio	86,219	95,257	9,038	92,097
urllib3	23,694	25,260	1,566	24,748

Table 4.10: M1 snapshot-size statistics by project.

The expanded dataset is intentionally heterogeneous. Large projects such as junit5, curl, and brew dominate absolute churn and line-count totals, while

4.5. DETAILED DESCRIPTIVE RESULTS FOR M1–M4

smaller libraries such as `requests`, `flask`, and `click` provide a continuity check against the pilot workflow. For this reason, Phase II should rely on normalized and within-project views when comparing candidate metric behavior.

4.5 DETAILED DESCRIPTIVE RESULTS FOR M1–M4

This section reports the descriptive outputs computed from the centralized Phase I dataset. The tables are not used to rank metrics or select a proof-of-concept metric. They document the empirical basis for the Phase II comparison under the predefined criteria.

4.5.1 M1 SNAPSHOT SIZE EVOLUTION

Project	First code	Last code	Delta	Delta %	Min	Max
brew	213073	220989	7916	3.72	213073	220989
click	13454	16504	3050	22.67	13454	16504
curl	224573	225738	1165	0.52	219978	225738
flask	12586	12977	391	3.11	12586	12977
gin	13510	17974	4464	33.04	13510	17974
junit5	132705	132689	-16	-0.01	132615	133150
prettier	112668	113217	549	0.49	112668	113217
requests	7964	8412	448	5.63	7964	8425
tokio	86219	95257	9038	10.48	86219	95385
urllib3	23694	25260	1566	6.61	23694	25260

Table 4.11: M1 code-line evolution across frozen tags. First and last values follow chronological tag order.

M1 captures source-size change over the frozen release interval. The largest relative growth appears in `gin`, `click`, and `tokio`. `junit5` is effectively stable in total code-line size across its selected tags, even though its windows contain substantial churn.

4.5.2 M2 CHURN AND M3 CADENCE INTENSITY

Project	Human commits	Human churn	Churn/commit	Commits/day	Churn/day	Mean authors/window
brew	311	39565	127.22	8.904	1132.80	11.29
click	367	28901	78.75	0.431	33.96	14.29
curl	3229	420253	130.15	9.610	1250.74	27.57
flask	156	12935	82.92	0.179	14.82	6.43
gin	295	20943	70.99	0.216	15.37	25.29
junit5	3386	1455032	429.72	16.268	6990.58	16.14
prettier	76	8117	106.80	0.549	58.58	2.43
requests	125	3115	24.92	0.149	3.72	8.43
tokio	505	55496	109.89	4.853	533.33	32.71
urllib3	111	12408	111.78	0.230	25.73	7.29

Table 4.12: M2/M3 human-only project-level intensity.

The centralized dataset has a wider activity range than the pilot. `curl` and `junit5` carry high commit and churn volume, while `requests`, `flask`, and `urllib3` provide lower-volume contrast. These differences are useful for Phase II because a candidate metric should remain interpretable across both high- and low-activity projects.

4.5.3 BOT-SENSITIVITY OUTPUTS

Project	Bot rows	Bot commit %	Bot churn %	Windows with bots	Bot top-commit windows
brew	10	17.72	9.10	5	0
click	8	13.03	0.94	6	1
curl	10	3.78	0.33	6	0
flask	2	13.33	1.70	1	0
gin	5	25.32	5.15	5	5
junit5	7	14.10	0.12	7	0
prettier	4	6.17	0.94	3	0
requests	3	17.76	4.94	3	2
tokio	1	0.20	0.00	1	0
urllib3	5	25.00	2.19	5	1

Table 4.13: Automation impact summary. Percentages are relative to all-account project totals.

Across the centralized dataset, bot rows contribute 999 of 9560 commits (10.45%) but only 9294 of 2,066,059 churn lines (0.45%). Automation therefore matters most for cadence and ownership interpretation, not for aggregate churn

4.5. DETAILED DESCRIPTIVE RESULTS FOR M1–M4

volume. The primary M3 and M4 evidence uses human-only outputs, while all-account outputs remain available for audit and sensitivity checks.

4.5.4 DURATION-NORMALIZED ANALYSIS OUTPUTS

Sensitivity thresh- old	Window count	Affected windows
<1 day	9	requests_W03; requests_W04; brew_W02; junit5_W02; junit5_W04; junit5_W06; prettier_W02; tokio_W01; tokio_W04
<7 days	22	requests_W03; requests_W04; urllib3_W05; urllib3_W06; brew_W02; brew_W03; brew_W04; brew_W05; brew_W06; junit5_W02; junit5_W04; junit5_W06; prettier_W01; prettier_W02; prettier_W03; prettier_W05; prettier_W07; tokio_W01; tokio_W04; tokio_W05; tokio_W06; tokio_W07
<14 days	27	The previous windows plus click_W04, requests_W05, brew_W01, brew_W07, and curl_W01.

Table 4.14: Short-window sensitivity flags. Windows below seven days are retained but marked as `sensitivity_only`.

Window	Duration days	Commits/day	Churn/day	Analysis set
junit5_W06	0.030	26909.313	10006801.83	<code>sensitivity_only</code>
junit5_W04	0.034	23433.962	8822406.47	<code>sensitivity_only</code>
junit5_W02	0.135	4976.771	375831.47	<code>sensitivity_only</code>
tokio_W01	0.577	277.240	37366.70	<code>sensitivity_only</code>
tokio_W04	0.854	291.516	31412.33	<code>sensitivity_only</code>
junit5_W05	39.870	7.399	6573.49	<code>primary</code>

Table 4.15: Highest human-only churn/day windows. Very short windows can dominate per-day rates, so they are not used as primary cadence evidence.

The normalized artifacts provide aggregate per-day commit and churn rates, per-KLOC-day churn rates, and parallel developer-window fields. Raw totals remain useful descriptive evidence, while duration-sensitive comparisons should use the normalized tables together with the `analysis_set` flag.

4.5.5 M3 DEVELOPER ACTIVITY-DAY PROFILE

Project	Mean active days/dev-window	Min	Max	Min window days	Max window days
brew	1.90	1	7	0.83	8.96
click	2.10	1	19	10.04	489.99
curl	5.04	1	58	7.00	63.00
flask	1.84	1	17	16.05	219.96
gin	1.50	1	12	60.82	378.26
junit5	6.74	1	95	0.03	69.91
prettier	1.59	1	4	0.87	78.36
requests	1.54	1	8	0.26	376.04
tokio	1.92	1	25	0.58	58.99
urllib3	1.75	1	13	3.00	170.05

Table 4.16: M3 human-only activity-day profile.

The activity-day profile confirms that cadence is sensitive to both project rhythm and release-window construction. This is why Phase II should compare cadence-related indicators with both full-data and robust, short-window-filtered views.

4.5.6 M4 OWNERSHIP AND CONCENTRATION PROFILE

Project	Top1 commits	Top3 commits	Top1 churn	Top3 churn	Gini commits	Gini churn
brew	0.35	0.68	0.64	0.86	0.40	0.71
click	0.36	0.69	0.50	0.86	0.51	0.72
curl	0.46	0.88	0.54	0.97	0.81	0.89
flask	0.71	0.89	0.89	0.99	0.49	0.68
gin	0.33	0.52	0.48	0.73	0.23	0.57
junit5	0.79	0.96	0.90	0.98	0.87	0.91
prettier	0.87	0.96	0.83	0.99	0.35	0.32
requests	0.57	0.79	0.71	0.94	0.22	0.41
tokio	0.30	0.61	0.42	0.71	0.31	0.56
urllib3	0.63	0.78	0.69	0.94	0.31	0.50

Table 4.17: Mean human-only M4 concentration indicators.

M4 shows substantial variation in human concentration. Some projects have broad participation by author count but still concentrate churn in a small set of contributors. This distinction is important for Phase II because ownership-style

4.6. PHASE I EXECUTION LEDGER

metrics may reward sustained maintainership differently from simple activity-volume metrics.

4.6 PHASE I EXECUTION LEDGER

This section records the operational sequence executed for the centralized Phase I dataset. The ledger links each step to its primary artifact and gives the validation result that makes the artifact usable in later analysis.

CHAPTER 4. DATASET CONSTRUCTION AND DESCRIPTIVE RESULTS

Step	Action	Primary artifacts	Verification result
2.1	Freeze repository meta-data	data/dataset_freeze_all.csv	10 repositories frozen with cohort labels, repository URLs, persistence status, and software-type tags.
2.2	Freeze stable release tags	data/versions_all.csv; data/version_tag_rules_all.csv	80 stable tags frozen, eight per repository, with each tag tied to an immutable commit hash.
2.3	Build consecutive windows	data/windows_all.csv	70 tag-to-tag windows generated in chronological order with no chain or chronology errors.
2.4.1	Extract raw developer-window metrics	out/metrics_window_dev_all.csv; out/metrics_window_totals_all.csv	1,118 developer-window rows and 70 window-total rows produced; extraction error logs remained empty.
2.4.2	Clean and validate developer rows	out/metrics_window_dev_clean_all.csv; out/metrics_window_totals_clean_all.csv; logs/clean_all_metrics.log	Clean rows remained 1,118; duplicate groups merged were 0; zero-commit and zero-churn windows were 0.
2.4.3	Compute LOC/SLOC snapshots (M1)	out/loc_per_tag_all.csv; data/loc_language_policy_all.csv; logs/loc_full_recompute_20260508.log	80 per-tag rows produced using the centralized multi-language source policy; full recomputation produced 80 rows with 0 mismatches.
2.4.4	Compute ownership/concentration (M4)	out/ownership_dev_shares_clean_all.csv; out/ownership_summary_clean_all.csv	1,118 all-account developer-share rows and 70 summary rows generated from clean data.
2.4.5	Separate automation and human-only evidence	out/metrics_window_dev_human_all.csv; out/ownership_summary_human_all.csv; out/bot_summary_by_project_all.csv	1,063 human developer-window rows, 70 human ownership summaries, and automation summaries for 6 bot identities generated.
2.4.6	Normalize for uneven window duration	out/metrics_window_totals_human_normalized_all.csv; out/metrics_window_dev_human_normalized_all.csv; out/window_duration_flags_all.csv	Per-day and per-KLOC-day fields generated; 22 windows below seven days marked as <code>sensitivity_only</code> .
2.4.7	Validate canonical repository layout	logs/canonical_repos_validation.log	10 colocated repositories checked; 80 frozen tags resolved to their frozen commit hashes; 0 missing or mismatched tags/commits.
2.4.8	Remove superseded pilot-only files	logs/pilot_cleanup_20260505.log	32 pilot-only CSV/log files removed after verification that the active evidence is centralized in the <code>_all</code> artifacts.
2.4.9	Remove superseded extended/scoped files	logs/centralized_cleanup_20260508.log	42 extended CSV/log files and 8 duplicate scoped output folders removed after subset and rerun verification against the centralized <code>_all</code> artifacts.

Table 4.18: Execution ledger for the centralized Phase I pipeline.

4.7. REPRODUCIBILITY PROTOCOL AND DATA LINEAGE

4.6.1 CLEANING AND VALIDATION OUTCOMES

The cleaning and recomputation logs provide concrete numerical guarantees:

- input developer-window rows: 1,118; cleaned rows: 1,118;
- duplicate key groups merged during cleaning: 0;
- windows with zero churn: 0; windows with zero commits: 0;
- bad windows for share-sum checks: 0;
- bad windows for unique-author consistency: 0;
- human-only rows generated for primary M3/M4 analysis: 1,063;
- bot-flagged rows retained for audit: 55;
- bot contribution: 999 commits (10.45%) and 9294 churn lines (0.45%);
- duration-normalized rows generated: 70 aggregate rows and 1,118 developer rows for the all-account layer, plus 70 aggregate rows and 1,063 developer rows for the human-only layer;
- windows shorter than seven days and flagged as `sensitivity_only`: 22.

These outcomes confirm that the centralized Phase I artifacts are internally consistent and ready to serve as auditable input for Phase II.

4.7 REPRODUCIBILITY PROTOCOL AND DATA LINEAGE

4.7.1 ARTIFACT LAYOUT

Phase I artifacts are organized under a single workspace root, `Data, Outputs, Logs, Scripts/Phase1/`, with fixed subdirectories: `data/`, `repos/`, `scripts/`, `out/`, and `logs/`. The final evidence layer uses the `_all` suffix. Pilot-only and extended-only CSVs have been consolidated into the centralized files and removed from the active evidence layer. The centralized `_all` files serve as the inputs for Phase II and for the Phase III metric freeze.

All ten Git working copies are colocated under the canonical `repos/` directory. Each repository uses the official GitHub URL from `data/dataset_freez`

`e_all.csv` as its origin. The layout has also been validated against `data/versions_all.csv`: all 80 frozen tags resolve to the frozen commit hashes in the centralized repository folder. The validation record is stored in `logs/canonical_repos_validation.log`.

4.7.2 EXECUTION PROTOCOL

The pipeline is deterministic given the frozen version and window files. A full rerun follows this order:

1. freeze repository metadata in `data/dataset_freeze_all.csv`;
2. freeze stable tags and commit hashes in `data/versions_all.csv`;
3. build consecutive tag-to-tag windows in `data/windows_all.csv`;
4. run `scripts/extract_window_metrics.py` to generate developer-window and window-total rows;
5. run `scripts/clean_round1_metrics.py` to canonicalize developer rows and recompute clean totals;
6. run `scripts/compute_loc_per_tag_multilang.py` for M1 snapshot-size evidence;
7. run `scripts/recompute_ownership_from_clean.py` for M4 concentration evidence;
8. run `scripts/compute_bot_sensitivity_outputs.py` to export human-only and bot-sensitivity tables;
9. run `scripts/compute_duration_normalized_metrics.py` for per-day, per-KLOC-day, and short-window flags;
10. verify row counts and validation logs before using the outputs in Phase II.

4.7. REPRODUCIBILITY PROTOCOL AND DATA LINEAGE

4.7.3 LINEAGE CHECKPOINTS

Artifact	Rows	Role
data/dataset_freeze_all.csv	10	Repository metadata and cohort membership.
data/version_tag_rules_all.csv	10	Stable-tag selection rules for all repositories.
data/loc_language_policy_all.csv	10	Source-extension policy for M1 LOC/SLOC counting.
data/versions_all.csv	80	Frozen tags and commit hashes (10 projects $\times$ 8 tags).
data/windows_all.csv	70	Consecutive windows (10 projects $\times$ 7 windows).
out/metrics_window_dev_clean_all.csv	1,118	Validated all-account developer-window dataset.
out/metrics_window_totals_clean_all.csv	70	All-account window totals from clean developer rows.
out/metrics_window_dev_human_all.csv	1,063	Human-only developer-window dataset for primary M3/M4 analysis.
out/metrics_window_totals_human_all.csv	70	Human-only window totals.
out/loc_per_tag_all.csv	80	M1 snapshot-size values per frozen tag.
out/ownership_dev_shares_human_all.csv	1,063	Human-only ownership shares.
out/ownership_summary_human_all.csv	70	Human-only concentration statistics for M4.
out/metrics_window_totals_human_normalized_all.csv	70	Human-only duration-normalized aggregate metrics.
out/metrics_window_dev_human_normalized_all.csv	1,063	Human-only duration-normalized developer metrics.
out/window_duration_flags_all.csv	70	Short-window flags and primary/sensitivity labels.
out/bot_summary_by_project_all.csv	10	Project-level automation contribution summary.
out/bot_summary_by_window_all.csv	70	Window-level automation contribution and top-contributor flags.
out/bot_identity_summary_all.csv	6	Automation identity summary.
logs/canonical_repos_validation.log	1	Canonical repository-layout validation log.
logs/loc_full_recompute_20260508.log	1	Full M1 LOC recomputation audit log; recomputed rows matched out/loc_per_tag_all.csv exactly.
logs/pilot_cleanup_20260505.log	1	Cleanup record for pilot-only files removed after consolidation.
logs/centralized_cleanup_20260508.log	1	Cleanup and verification record for extended-only and scoped duplicate files removed after consolidation.

Table 4.19: Centralized Phase I data lineage checkpoints used for reproducibility and auditing.

4.7.4 AUTOMATED VALIDATION EVIDENCE

Validation evidence is recorded in log files and confirms that:

- duplicate-key groups merged in cleaning: 0;
- windows with zero total churn: 0;
- windows with zero total commits: 0;
- windows failing share-sum validations: 0;
- windows failing unique-author consistency checks: 0;
- all-account total mismatches found during bot-sensitivity validation: 0;
- windows shorter than seven days and flagged as `sensitivity_only`: 22;
- bot rows retained for audit: 55; human-only rows exported for primary M3/M4 analysis: 1,063;
- canonical repository-layout validation: 10 repositories checked, 80 frozen tag rows checked, 0 missing or mismatched commits, and 0 missing or mismatched tags.
- pilot-only cleanup: 32 superseded pilot CSV/log files removed after their evidence was consolidated into the centralized `_all` layer.
- extended/scoped cleanup: 42 superseded extended CSV/log files and 8 duplicate scoped output folders removed after verification against the centralized `_all` layer.

These checks establish that downstream Phase II analysis operates on a coherent and fully auditable dataset.

4.8 PHASE I CONCLUSION

Phase I is complete at the centralized dataset and pipeline level. The thesis now has:

- a frozen 10-repository dataset with cohort labels and repository metadata;
- 80 frozen stable tags tied to immutable commit hashes;
- 70 tag-chronological release windows for tag-to-tag measurement, later qualified by the ancestry audit;

4.8. PHASE I CONCLUSION

- extracted and cleaned developer-window metrics for M1–M4;
- ownership and concentration outputs computed strictly from canonical clean data;
- human-only M3/M4 outputs and bot-sensitivity summaries;
- duration-normalized outputs and short-window sensitivity flags for uneven release windows;
- validation logs showing arithmetic, identity, aggregation, and ownership consistency;
- a DESTINO-aligned evidence boundary and a PoC integration blueprint later instantiated in Phase III.

Within Phase I, the work remains intentionally descriptive and methodological. No PoC metric is selected at this phase boundary. Selection is deferred to Phase II after comparative evaluation under the criteria in Section 3.1 and feasibility scoring in Section 3.2.1.

5

Comparative Evaluation and Metric Selection

5.1 PHASE II STEP 1: C1 STABILITY ANALYSIS

Phase II starts with the most empirical comparison criterion: stability. The purpose of this step is to measure how much each candidate signal changes across release windows and whether developer rankings remain similar from one window to the next. This step does not select the PoC metric. It produces the first evidence layer used later together with interpretability, gaming resistance, long-term suitability, and smart-contract feasibility.

5.1.1 ANALYSIS INPUTS AND GENERATED ARTIFACTS

The C1 script reads only the centralized Phase I _all evidence. It writes derived Phase II artifacts under `Data, Outputs, Logs, Scripts/Phase2/`, keeping comparative-analysis outputs separate from the frozen dataset and extraction layer.

5.1. PHASE II STEP 1: C1 STABILITY ANALYSIS

Artifact	Rows	Role
out/c1_metric_window_values_all.csv	350	Long-format window-level values for five stability signals across 70 windows.
out/c1_metric_stability_by_project_all.csv	50	Per-project volatility statistics for each stability signal.
out/c1_metric_stability_overall_all.csv	5	Cross-project summary of window-level volatility.
out/c1_rank_input_values_all.csv	3189	Developer-level values used for adjacent-window rank comparison.
out/c1_rank_stability_pairs_all.csv	180	Adjacent-window rank correlations for M2, M3, and M4 developer-level signals.
out/c1_rank_stability_by_project_all.csv	30	Per-project rank-stability summaries.
out/c1_rank_stability_overall_all.csv	3	Cross-project rank-stability summary.
logs/c1_stability_20260508.log	1	Execution log for the C1 stability run.

Table 5.1: Phase II C1 stability artifacts. Paths are relative to the Phase II analysis directory.

5.1.2 METHOD

For window-level stability, five signals are compared:

- M1 endpoint KLOC, using the code-line count at each window’s target tag;
- M2 churn per KLOC-day, using size- and duration-normalized churn;
- M3 commits per day, using duration-normalized activity cadence;
- M4 churn Gini, using contributor concentration in churn;
- M4 top-1 churn share, using the largest contributor’s churn share.

For each project and signal, the script computes the coefficient of variation and adjacent-window relative change. The same statistics are computed twice: once on all windows, and once on the robust subset that excludes windows marked `sensitivity_only`. For developer-level rank stability, the script compares adjacent windows using Spearman rank correlation among developers who appear in both windows. Rank stability is computed for M2 developer churn/day, M3 developer commits/day, and M4 developer churn share.

5.1.3 WINDOW-LEVEL STABILITY RESULTS

Signal	Median CV	Primary CV	Primary adjacent change	Primary high-vol. projects
M1 endpoint KLOC	0.013	0.014	0.009	0
M2 churn/KLOC/day	1.208	0.890	0.706	6
M3 commits/day	0.876	0.608	0.644	4
M4 churn Gini	0.424	0.329	0.337	2
M4 top-1 churn share	0.289	0.236	0.208	1

Table 5.2: C1 window-level volatility summary. CV is the coefficient of variation across project windows; “primary” excludes short windows marked `sensitivity_only`.

M1 exhibits the lowest observed volatility because project size changes slowly between adjacent stable releases. That stability makes it a useful context and normalization signal, but it does not directly measure developer contribution quality or reputation. M2 and M3 are much more dynamic. Churn is especially volatile, which is expected because release content can vary sharply even inside the same project. M4 concentration signals are more stable than raw activity measures, with top-1 churn share showing the lowest volatility among behavior-oriented signals in this pass.

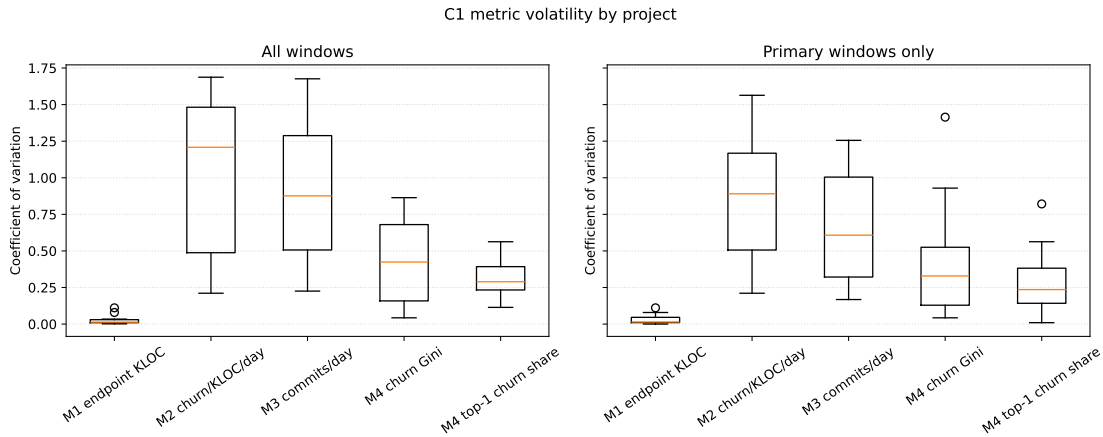


Figure 5.1: Distribution of per-project coefficient of variation for each C1 window-level signal.

5.1. PHASE II STEP 1: C1 STABILITY ANALYSIS

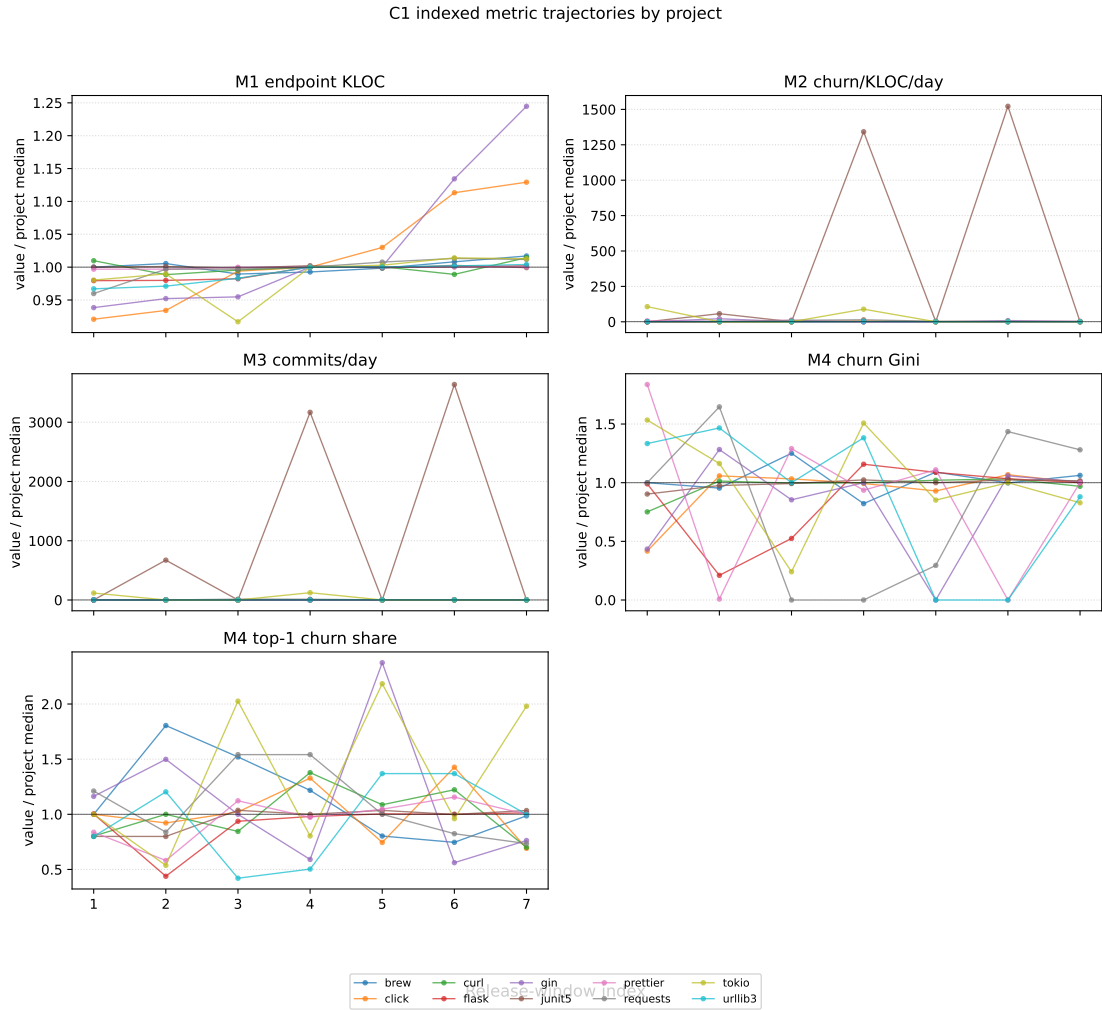


Figure 5.2: Indexed C1 signal trajectories by project. Each line is scaled by that project’s median value for the signal.

5.1.4 DEVELOPER-RANK STABILITY RESULTS

Developer-level signal	Median ρ	Primary ρ	Common share	Valid pairs
M2 developer churn/day	0.668	0.608	0.172	36/60
M3 developer commits/day	0.858	0.836	0.172	33/60
M4 developer churn share	0.668	0.608	0.172	36/60

Table 5.3: C1 developer-rank stability summary across adjacent windows. Correlations use only developers appearing in both adjacent windows.

The rank-stability result should be read together with the common-developer share. The median overlap between adjacent windows is only 0.172, so many

adjacent windows involve a partially different contributor set. Among developers who do recur, M3 commit cadence has the strongest rank continuity. M2 churn/day and M4 churn share produce the same rank-stability values because churn share is a within-window normalization of churn; it changes the unit and interpretation, but not the developer ordering inside the same window.

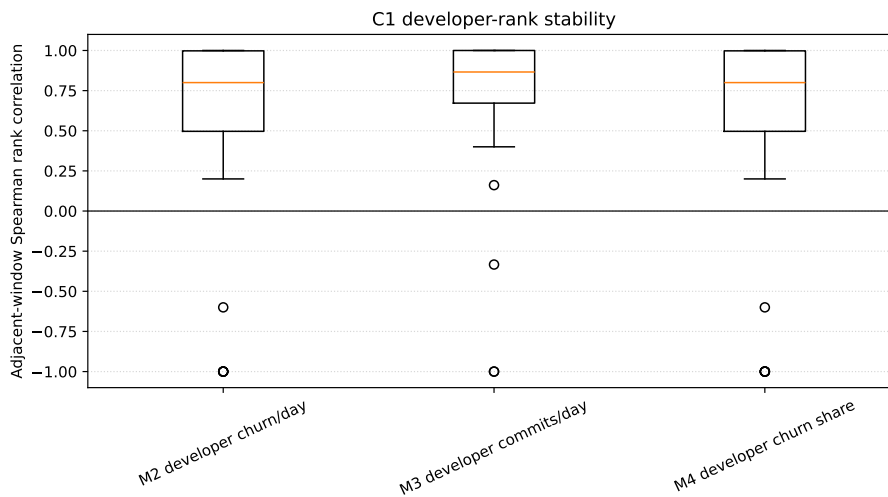


Figure 5.3: Adjacent-window developer-rank stability for M2, M3, and M4 developer-level signals.

5.1.5 C1 INTERPRETATION FOR THE DECISION PROCESS

The C1 evidence does not select a final PoC metric, but it narrows the interpretation of the candidates:

- M1 is empirically stable but mainly useful as context and normalization support.
- M2 is informative about release effort but too volatile to use alone without safeguards.
- M3 has better developer-rank continuity than M2, but still depends on release cadence and contributor overlap.
- M4 concentration signals are comparatively stable and remain promising for reputation interpretation, but they require careful gaming-resistance analysis.

Table 5.2 and the figures in this section retain the original 70-window baseline. Section 5.2 audits whether those tag-chronological windows are also linear

5.2. POST-HOC RELEASE-WINDOW ANCESTRY VALIDATION

Git intervals and reports the separate ancestry-only C1 sensitivity result before the remaining decision criteria are interpreted.

The next section evaluates C2 interpretability using the same candidate set and the C1 findings. After C2, C3 gaming resistance and C4 long-term suitability can be scored before applying the smart-contract feasibility rubric.

5.2 POST-HOC RELEASE-WINDOW ANCESTRY VALIDATION

The original freeze orders stable tags by tag date and forms adjacent chronological pairs. This rule is reproducible, but chronological adjacency does not guarantee Git ancestry when a project maintains parallel release lines or publishes backports. For a window represented by `FROM..TO`, the extracted commit set is a sequential development interval only if `FROM` is an ancestor of `TO`. The original artifacts are preserved, while this post-hoc check adds a separate validity and sensitivity layer.

5.2.1 AUDIT METHOD AND IMPACT

All 70 frozen windows were checked with `git merge-base --is-ancestor`. Fifty-eight windows pass and 12 are non-ancestral. The affected set contains one Flask window, two Gin windows, all seven JUnit windows, one Prettier window, and one Tokio window. JUnit alternates between the 5.14 and 6.0 release lines, so none of its original adjacent tag-date pairs is retained in the ancestry-only subset.

Audit quantity	Result
Frozen tag-chronological windows	70
Ancestor-to-descendant windows	58
Non-ancestral windows	12
Projects with at least one non-ancestral window	5
Human commits in non-ancestral windows	3,576 / 8,561 (41.77%)
Human churn in non-ancestral windows	1,471,597 / 2,056,765 (71.55%)

Table 5.4: Post-hoc Git ancestry audit and quantitative impact.

For a non-ancestral pair, `FROM..TO` is still a deterministic Git set difference, but it is not the development performed between two releases on one linear his-

tory. This distinction materially affects interpretation because the 12 affected windows contain most of the original human churn. They are therefore excluded from the sensitivity analysis and from baseline PoC eligibility, rather than silently treated as ordinary release intervals.

5.2.2 ANCESTRY-ONLY C1 SENSITIVITY ANALYSIS

The sensitivity analysis imports the frozen official C1 implementation, so metric definitions, primary-window flags, coefficients of variation (CV), and volatility thresholds are unchanged. It filters only by the ancestry audit and writes new artifacts outside the Phase I and Phase II directories. Nine projects retain at least one window; eight projects retain at least two primary windows and can contribute to the primary cross-project summary.

Signal	Original primary CV	Ancestral primary CV	Original high-vol.	Ancestral high-vol.
M1 endpoint KLOC	0.014392	0.014392	0/10	0/8
M2 churn/KLOC/day	0.890454	0.890454	6/10	5/8
M3 commits/day	0.607560	0.631179	4/10	3/8
M4 churn Gini	0.328938	0.351228	2/10	1/8
M4 top-1 churn share	0.235787	0.246522	1/10	0/8

Table 5.5: Original and ancestry-only C1 window-stability results. High-volatility counts use only projects with at least two primary windows.

M1 remains the most stable context signal but is not eligible as standalone developer-reputation evidence. Among the implemented behavioral candidates, the qualitative ordering is retained: both M4 window-concentration signals remain less volatile than M2 and M3. M4 top-one churn share remains within the frozen low-volatility threshold ($CV \leq 0.25$), while M4 churn Gini remains moderate. This result supports retaining M4 for the engineering PoC; it does not retroactively turn the 12 excluded windows into valid linear intervals or establish universal superiority.

5.2.3 REPRODUCIBILITY ARTIFACTS

The additive package under `Data, Outputs, Logs, Scripts/PosthocValidation/` contains the 70-row ancestry audit, per-project and global impact summaries, the 58-window/290-signal-row sensitivity dataset, the original-versus-sensitivity comparison, deterministic logs, and scripts. An identical rerun pro-

5.3. PHASE II STEP 2: C2 INTERPRETABILITY ANALYSIS

duced byte-identical CSV outputs. The original Phase I and Phase II artifacts remain unchanged and are retained as the documented baseline analysis.

5.3 PHASE II STEP 2: C2 INTERPRETABILITY ANALYSIS

The second Phase II criterion evaluates whether each candidate metric can be explained clearly and audited without relying on hidden assumptions. This step is deliberately separated from C1 stability: a metric can be stable but weak as a reputation signal, or meaningful but harder to explain. The C2 evidence therefore scores interpretability, not final suitability.

5.3.1 SCORING PROTOCOL AND ARTIFACTS

C2 uses a fixed five-dimension rubric. Each dimension is scored on a 0–2 scale: **0** means weak, **1** means partial, and **2** means clear. The maximum score is 10. The dimensions C2_D1–C2_D5 are local to the C2 interpretability criterion; they are not the same as the later feasibility dimensions D1–D10. The rubric is generated as a CSV artifact so that the score basis remains explicit.

The numerical C2 ratings are structured author assessments encoded as constants with written rationales. The script reproduces the rubric and totals; it does not infer interpretability from repository observations or provide independent rater agreement. C2 is therefore a transparent decision aid rather than an empirical measurement.

Dimension	Name	Meaning
C2_D1	Semantic clarity	Whether the metric has a plain and stable meaning.
C2_D2	Unit clarity	Whether the unit can be explained without ambiguity.
C2_D3	Normalization clarity	Whether normalization improves comparison without obscuring meaning.
C2_D4	Tooling transparency	Whether extraction can be reproduced from explicit inputs.
C2_D5	Reputation explanation fit	Whether the metric can support a developer-reputation explanation.

Table 5.6: C2 interpretability rubric. Each dimension is scored from 0 to 2.

Artifact	Rows	Role
out/c2_interpretability_rubric_all.csv	5	Fixed C2 scoring dimensions and definitions.
out/c2_interpretability_scores_all.csv	25	Per-metric, per-dimension scores with written rationales.
out/c2_interpretability_summary_all.csv	5	Metric-level total score, band, strongest dimensions, and weakest dimensions.
logs/c2_interpretability_20260508.log	1	Execution log for the C2 scoring run.

Table 5.7: Phase II C2 interpretability artifacts. Paths are relative to the Phase II analysis directory.

5.3.2 C2 RESULTS

ID	Metric	Status	Score	Band	Interpretation
M3	Activity cadence	Implemented	9/10	High	Clearest activity-continuity signal; still should not be treated as quality by itself.
M1	Snapshot size	Implemented	8/10	High	Straightforward to explain and reproduce, but mainly useful as context rather than reputation evidence.
M2	Churn	Implemented	8/10	High	Easy to explain as change magnitude, with normalization and quality caveats.
M4	Ownership / concentration	Implemented	7/10	Medium	Strong reputation relevance, but shares, Gini, and identity aggregation require careful explanation.
M5	Maintenance responsiveness	Design extension	6/10	Medium	Conceptually useful, but excluded from the current comparison decision because issue/PR event data were not collected.

Table 5.8: C2 interpretability summary for the candidate metrics.

5.3. PHASE II STEP 2: C2 INTERPRETABILITY ANALYSIS

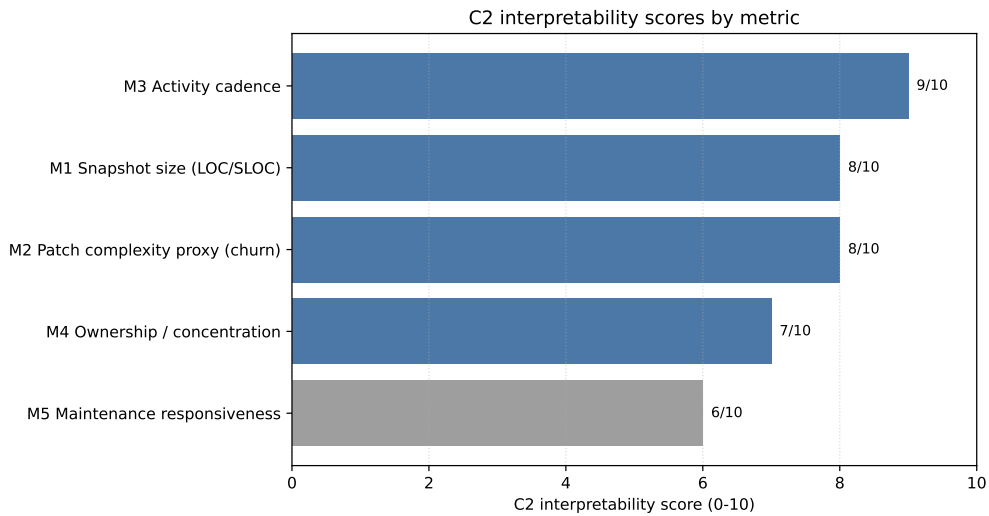


Figure 5.4: C2 interpretability scores on a 0–10 scale. The design-extension metric is shown separately from implemented metrics.

The C2 result strengthens the distinction between simple measurement and useful reputation evidence. M3 receives the highest interpretability score because commits, active days, and commits/day are easy to explain and the duration normalization is direct. However, the interpretation remains bounded: activity continuity is not the same as contribution quality. M1 and M2 also score highly because their units are transparent. Their weakness is not readability but meaning: M1 describes project size, and M2 describes change magnitude rather than quality.

M4 receives a lower interpretability score, but it remains important. Ownership and concentration are closer to maintainer responsibility, yet they require more explanation because Gini, top-contributor shares, and identity canonicalization are less immediate than counts or rates. M5 remains a design extension. Its concept is clear, but the current evidence base intentionally excludes issue and pull-request event data, so it should not compete with the implemented candidates in the current comparison decision.

5.3.3 C2 INTERPRETATION FOR THE DECISION PROCESS

C2 does not eliminate any implemented metric. Instead, it defines how each candidate should be discussed in the remaining comparison:

- M1 can support context and normalization claims, but not direct developer-reputation claims.

- M2 can support effort and change-magnitude claims, but it needs safeguards against reading churn as quality.
- M3 is the easiest implemented reputation-adjacent signal to explain, especially after duration normalization.
- M4 has the strongest conceptual link to maintainer influence, but it must be presented with clear definitions and identity rules.
- M5 should remain documented for future work and should not be selected for the baseline PoC unless new event data are added.

The next section evaluates C3 gaming resistance. That step focuses on how easily each metric can be manipulated and whether manipulation can be detected from the existing Phase I evidence.

5.4 PHASE II STEP 3: C3 GAMING-RESISTANCE ANALYSIS

The third Phase II criterion evaluates how easily each metric can be manipulated. This criterion is necessary because developer-reputation systems can create incentives to optimize the measured signal rather than the underlying maintenance behavior. The C3 scoring therefore asks whether gaming is difficult, costly, detectable, and controllable with the evidence already available in the centralized Phase I outputs.

5.4.1 SCORING PROTOCOL AND ARTIFACTS

C3 uses a fixed five-dimension rubric. Each dimension is scored on a 0–2 scale, where higher values mean stronger resistance to manipulation. A metric can receive a good C3 score even if it is not perfect, provided that likely manipulation is visible and can be controlled through cross-checks. The dimensions C3_D1–C3_D5 are local to the C3 gaming-resistance criterion; they are not the same as the later feasibility dimensions D1–D10.

The C3 scores are structured author assessments of documented attack surfaces and safeguards. No adversarial experiment, developer study, or manipulation simulation was performed. A high C3 score therefore means “judged comparatively stronger under this rubric,” not empirically proven resistance to gaming.

5.4. PHASE II STEP 3: C3 GAMING-RESISTANCE ANALYSIS

Dimension	Name	Meaning
C3_D1	Manipulation difficulty	How difficult it is to artificially improve the metric.
C3_D2	Manipulation cost	How much real work or coordination manipulation requires.
C3_D3	Detectability from current evidence	Whether manipulation can be flagged from existing Phase I artifacts.
C3_D4	Cross-metric consistency	Whether the metric can be checked against other candidate signals.
C3_D5	Residual-risk control	Whether remaining gaming risk is acceptable after safeguards.

Table 5.9: C3 gaming-resistance rubric. Each dimension is scored from 0 to 2, with higher scores meaning stronger resistance.

Artifact	Rows	Role
out/c3_gaming_rubric_all.csv	5	Fixed C3 scoring dimensions and definitions.
out/c3_gaming_attack_surface_all.csv	10	Metric-specific attack vectors, evidence fields, and safeguards.
out/c3_gaming_scores_all.csv	25	Per-metric, per-dimension scores with written rationales.
out/c3_gaming_summary_all.csv	5	Metric-level total score, band, strongest dimensions, and weakest dimensions.
logs/c3_gaming_resistance_20260508.log	1	Execution log for the C3 scoring run.

Table 5.10: Phase II C3 gaming-resistance artifacts. Paths are relative to the Phase II analysis directory.

5.4.2 ATTACK-SURFACE SUMMARY

The attack-surface matrix records likely manipulations and the current detection boundary. M1 can be inflated by adding low-value code or vendored/generated source. M2 can be inflated through unnecessary rewrites or formatting changes. M3 can be manipulated by splitting work into many commits or by automation-heavy activity. M4 is harder to manipulate because it depends on relative ownership over a window, but it can still be distorted by one large dominant change or by unresolved identity-linkage issues. M5 is conceptually attractive, but because the current evidence excludes issue and pull-request event streams, responsiveness gaming cannot be audited in this dataset.

5.4.3 C3 RESULTS

ID	Metric	Status	Score	Band	Interpretation
M4	Ownership / concentration	Implemented	9/10	High	Strongest profile under the C3 rubric, but still needs identity and dominance caveats.
M2	Churn	Implemented	7/10	Medium	Manipulable through unnecessary churn, but current evidence gives useful detection signals.
M3	Activity cadence	Implemented	6/10	Medium	Commit splitting is easy, so cadence needs churn, active-day, and bot cross-checks.
M1	Snapshot size	Implemented	5/10	Medium	Safe as context, weak as direct reputation evidence because size can be inflated.
M5	Maintenance responsiveness	Design extension	3/10	Low	Current evidence cannot detect responsiveness gaming; exclude from baseline PoC decision.

Table 5.11: C3 gaming-resistance summary for the candidate metrics.

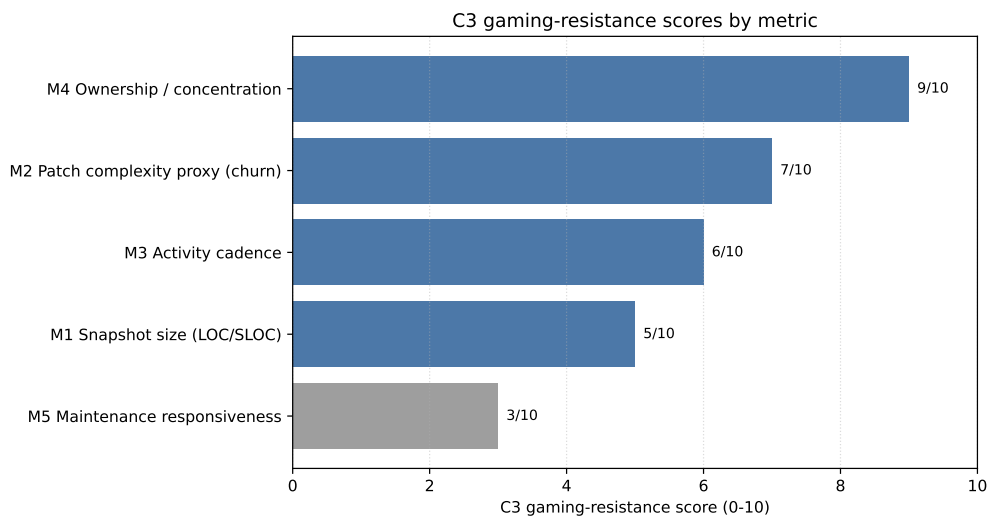


Figure 5.5: C3 gaming-resistance scores on a 0–10 scale. Higher values indicate stronger resistance after safeguards.

The C3 result changes the emphasis from C2. M3 was the most interpretable implemented metric, but it is easier to manipulate through commit splitting. M4 is harder to explain, but it has the strongest resistance profile because sustained ownership share is harder to fake than isolated commit counts or churn

5.5. PHASE II STEP 4: C4 LONG-TERM SUITABILITY ANALYSIS

spikes. M2 remains useful as effort evidence, but it should not be rewarded directly without checks against M3 cadence, M4 concentration, LOC movement, and short-window flags. M1 remains appropriate as a context or normalization metric, not as a standalone reputation score.

5.4.4 C3 INTERPRETATION FOR THE DECISION PROCESS

C3 does not select the PoC metric, but it introduces a strong warning against simple activity-based reputation:

- M1 should not be used directly for reputation because size is not maintainer quality.
- M2 is useful only when paired with safeguards against artificial churn.
- M3 is easy to explain but needs protection against commit splitting and bot-heavy activity.
- M4 is the strongest candidate under the current C3 rubric, provided identity handling and dominance spikes are documented.
- M5 remains excluded from the current comparison decision because the evidence needed to audit responsiveness is absent.

The next section evaluates C4 long-term suitability. That step checks whether each metric remains informative over longer histories and whether it can be updated incrementally without recomputing the full project history.

5.5 PHASE II STEP 4: C4 LONG-TERM SUITABILITY ANALYSIS

The fourth Phase II criterion evaluates whether each metric remains useful when the project history becomes longer. This is different from short-window stability. A metric can be stable in a few adjacent windows but still become less useful when the project grows, changes release rhythm, or moves from active development to maintenance. C4 therefore checks historical coverage, lifecycle robustness, incremental update feasibility, compact auditability, and long-term reputation fit.

5.5.1 SCORING PROTOCOL AND ARTIFACTS

C4 uses a five-dimension rubric. Each dimension is scored on a 0–2 scale, where higher values mean stronger long-term suitability. The scoring uses the centralized Phase I evidence for coverage and update feasibility, while the reputation-fit dimension records whether the metric remains meaningful as a long-term signal. The dimensions C4_D1–C4_D5 are local to the C4 long-term suitability criterion; they are not the same as the later feasibility dimensions D1–D10.

Historical coverage counts are derived from the dataset, while the dimension scores and long-term interpretations are structured author assessments. With seven original windows per project and heterogeneous history spans, C4 supports a comparative design judgment but does not establish multi-year predictive validity.

Dimension	Name	Meaning
C4_D1	Historical coverage	Whether the current evidence covers the metric across frozen projects and windows.
C4_D2	Lifecycle robustness	Whether the metric remains meaningful across growth, maintenance, and short-release phases.
C4_D3	Incremental update feasibility	Whether new releases can be added without recomputing the full history.
C4_D4	Compact audit record	Whether long-term evidence can be stored as compact, hash-anchored records.
C4_D5	Long-term reputation fit	Whether the metric remains useful for reputation reasoning over extended histories.

Table 5.12: C4 long-term suitability rubric. Each dimension is scored from 0 to 2.

Artifact	Rows	Role
out/c4_lifecycle_coverage_all.csv	10	Per-project historical span, window counts, primary-window counts, and sensitivity-window counts.
out/c4_metric_evidence_all.csv	5	Metric-level evidence coverage and analysis unit.
out/c4_longterm_rubric_all.csv	5	Fixed C4 scoring dimensions and definitions.
out/c4_longterm_scores_all.csv	25	Per-metric, per-dimension scores with written rationales.
out/c4_longterm_summary_all.csv	5	Metric-level total score, band, strongest dimensions, and weakest dimensions.
logs/c4_longterm_suitability_20260508.log	1	Execution log for the C4 scoring run.

Table 5.13: Phase II C4 long-term suitability artifacts. Paths are relative to the Phase II analysis directory.

5.5.2 HISTORICAL COVERAGE

The C4 coverage table confirms that the centralized dataset contains 10 projects, 80 frozen release tags, and 70 release windows. Of those 70 windows, 48 are primary-analysis windows and 22 are retained as `sensitivity_only` because their duration is short. Project histories are uneven: the shortest covered span is 34.9 days for `brew`, while the longest is 1362.9 days for `gin`. This unevenness is useful for C4 because it tests whether a metric still has a clear interpretation across both dense release cycles and longer maintenance histories.

The implemented metrics M1–M4 all have complete current coverage under their respective analysis units. M1 covers all 80 tag snapshots. M2 and M3 use 1,063 human-only project-window-developer rows. M4 uses the same human-only developer-share population plus window-level ownership summaries. M5 has no current empirical coverage because issue and pull-request event streams were intentionally excluded from the Phase I baseline.

5.5.3 C4 RESULTS

ID	Metric	Status	Score	Band	Interpretation
M3	Activity cadence	Implemented	8/10	High	Compact and easy to update, but lifecycle changes can alter what activity means.
M4	Ownership / concentration	Implemented	8/10	High	Strong long-term fit because relative ownership stays meaningful across project histories.
M1	Snapshot size	Implemented	7/10	Medium	Useful as long-term context and normalization support, not as direct reputation evidence.
M2	Churn	Implemented	7/10	Medium	Incremental and well-covered, but volatile and sensitive to refactoring style.
M5	Maintenance responsiveness	Design extension	4/10	Low	Conceptually useful, but not eligible for the current comparison decision without event data.

Table 5.14: C4 long-term suitability summary for the candidate metrics.

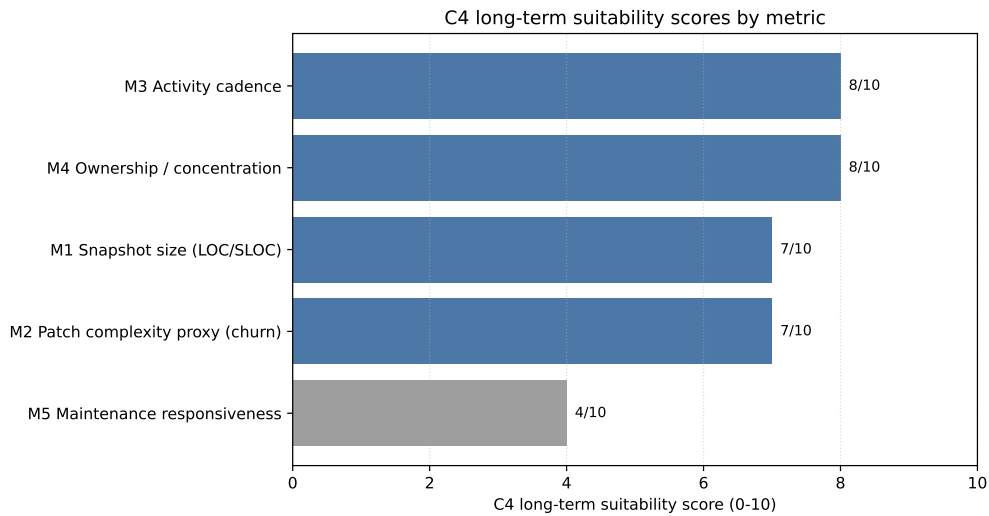


Figure 5.6: C4 long-term suitability scores on a 0–10 scale. Higher values indicate stronger suitability for long histories.

C4 does not reverse the earlier findings, but it clarifies their time horizon. M3 is strong for long-term operation because commit counts, active days, and normalized cadence can be appended window by window and stored compactly. The limitation is interpretive: activity naturally changes when a project matures, so cadence should not be read without churn and ownership context. M4 also scores high because relative ownership remains comparable even when projects differ in size and release rhythm. Its cost is operational rather than conceptual: the system must keep identity canonicalization and share calculations explicit.

M1 and M2 are useful supporting signals. M1 gives a durable size baseline and denominator, but it does not describe developer reputation by itself. M2 remains a practical measure of effort over time, but long histories make its volatility more visible, especially around refactors or formatting-heavy releases. M5 remains outside the baseline decision because there is no current issue or pull-request event evidence to update, audit, or validate.

5.5.4 C4 INTERPRETATION FOR THE DECISION PROCESS

C4 completes the four comparative criteria before smart-contract feasibility scoring:

- M3 is operationally attractive because it is compact and incremental, but it needs protection from the C3 gaming risks.

5.6. PHASE II STEP 5: SMART-CONTRACT FEASIBILITY SCORING

- M4 has the strongest long-term reputation fit among implemented metrics, but requires careful identity and state handling.
- M1 should remain in the baseline as contextual evidence and normalization support.
- M2 should remain available as effort evidence, especially when combined with cadence and ownership checks.
- M5 should stay documented for future work and should not enter the current PoC decision.

The next section applies the smart-contract feasibility scoring pass. That pass combines the C1–C4 findings with the predefined feasibility rubric before the final proof-of-concept metric is selected.

5.6 PHASE II STEP 5: SMART-CONTRACT FEASIBILITY SCORING

After completing C1–C4, the feasibility step tests whether the comparatively useful metrics can also work inside the DESTINO-aligned evidence model. This step applies the predefined smart-contract feasibility rubric from Section 3.2.1. The goal is not to move repository analysis on-chain. The intended PoC architecture remains hybrid: metric extraction is performed off-chain from hash-pinned evidence, while the contract stores compact values, provenance metadata, and evidence commitments.

This is a separate computation from C2, C3, and C4. Those criteria each score five criterion-specific dimensions on a 0–10 scale. The feasibility pass scores ten smart-contract dimensions, D1–D10, on a 0–20 scale.

5.6.1 SCORING PROTOCOL AND ARTIFACTS

The feasibility pass has two layers. First, it combines data-derived C1 evidence with structured C2–C4 assessments into one metric-comparison table. Second, it applies the ten-dimension feasibility rubric to each candidate. A metric is eligible for the final PoC selection gate only if it is implemented in the current evidence base, is suitable as a reputation signal, and does not trigger a feasibility exclusion condition.

Artifact	Rows	Role
out/phase2_metric_comparison_all.csv	5	Combined C1–C4 comparison scores and metric roles.
out/feasibility_rubric_all.csv	10	Fixed smart-contract feasibility dimensions.
out/feasibility_scores_all.csv	50	Per-metric, per-dimension feasibility scores and rationales.
out/feasibility_summary_all.csv	5	Metric-level feasibility totals, exclusions, and implementation accounts.
out/poc_candidate_decision_inputs_all.csv	5	Combined empirical and feasibility inputs for the final selection gate.
logs/feasibility_scoring_20260509.log	1	Execution log for the feasibility scoring run.

Table 5.15: Smart-contract feasibility scoring artifacts. Paths are relative to the Phase II analysis directory.

5.6.2 COMBINED C1–C4 COMPARISON

The combined comparison keeps the role of each metric explicit. M1 receives strong stability and feasibility scores, but it remains a context metric rather than a developer-reputation metric. M5 remains a design extension because the current dataset does not contain issue or pull-request event streams. The implemented reputation candidates are therefore M2, M3, and M4.

ID	Metric	C1	C2	C3	C4	Avg.	Role
M1	Snapshot size	10.0	8	5	7	7.50	Context only; useful for normalization, not standalone reputation.
M2	Churn	4.0	8	7	7	6.50	Eligible supporting candidate with volatility and quality caveats.
M3	Activity cadence	6.0	9	6	8	7.25	Eligible continuity candidate with commit-splitting safeguards.
M4	Ownership / concentration	8.5	7	9	8	8.13	Eligible reputation-oriented candidate with identity caveats.
M5	Maintenance responsiveness	0.0	6	3	4	3.25	Design extension excluded from the current comparison decision.

Table 5.16: Combined C1–C4 decision inputs. C1 is derived from primary high-volatility project counts; C2–C4 use the scored summaries from the previous sections.

Table 5.16 shows the comparison trade-off. M4 has the strongest implemented reputation-oriented profile because it combines better C1 stability than M2 and M3 with the strongest C3 gaming-resistance result and high C4 long-term suitability. M3 remains a strong alternative because it is easy to explain and compact over long histories, but C3 showed that activity cadence

5.6. PHASE II STEP 5: SMART-CONTRACT FEASIBILITY SCORING

is easier to manipulate. M2 remains useful evidence of effort, but the C1 result shows that churn is too volatile to use alone.

5.6.3 FEASIBILITY RESULTS

ID	Metric	Role	Score	Band	Excl.	Interpretation
M1	Snapshot size	Context only	18/20	High	None	Technically feasible, but not a standalone reputation metric.
M2	Churn	Eligible	16/20	High	None	Feasible as a release-window aggregate with hash-pinned replay evidence.
M3	Activity cadence	Eligible	17/20	High	None	Strong feasibility because cadence values are compact and release-window based.
M4	Ownership / concentration	Eligible	16/20	High	None	Feasible in a hybrid model, with identity canonicalization kept explicit.
M5	Responsiveness	Excluded	10/20	Medium	A; C	Not eligible because current evidence cannot replay or verify issue/PR events.

Table 5.17: Smart-contract feasibility summary. Exclusion labels follow Section 3.2.1.

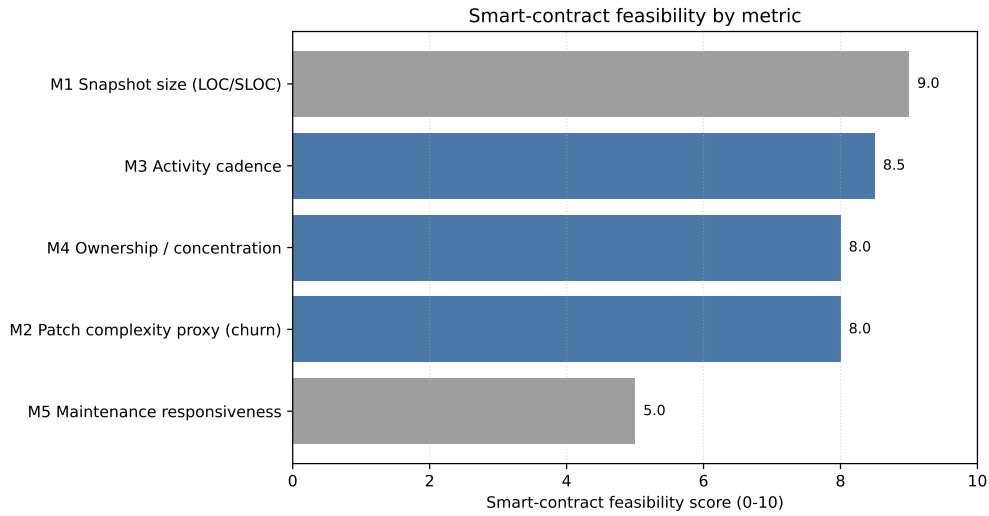


Figure 5.7: Smart-contract feasibility scores on a 0–10 scale. Grey bars indicate metrics that are not eligible for the current PoC decision.

The feasibility scores support the hybrid architecture defined earlier in the report. None of M2, M3, or M4 requires the contract to traverse a repository or recompute a metric on-chain. Their feasible implementation approach is to compute the metric off-chain from frozen, hash-pinned evidence, then store a compact metric value and evidence hash on-chain. The main differences are in storage shape and identity handling. M3 is the most compact eligible candidate. M4 is more reputation-relevant, but requires careful handling of developer identity and fixed-point representation for shares or concentration values.

5.6.4 FINAL SELECTION INPUTS

The final decision-input table combines the C1–C4 comparison average with the feasibility score. The generated CSV retains the historical column name `empirical_average_score`, although only C1 is directly data-derived. For ranking eligible metrics, the analysis uses a transparent weighting: 60% comparison average and 40% smart-contract feasibility. This weighting is not meant to hide the individual criteria; it is only a compact ordering tool after the separate scores have been reported.

5.6. PHASE II STEP 5: SMART-CONTRACT FEASIBILITY SCORING

ID	Metric	C1–C4 avg.	Feas.	Combined	Rank	Selection status
M4	Ownership / concentration	8.13	8.0	8.08	1	Preferred for the final selection gate.
M3	Activity cadence	7.25	8.5	7.75	2	Eligible alternative.
M2	Churn	6.50	8.0	7.10	3	Eligible alternative and supporting evidence.
M1	Snapshot size	7.50	9.0	8.10	–	Not selected as a standalone metric because it is context only.
M5	Responsiveness	3.25	5.0	3.95	–	Not selected because current evidence is absent and exclusions are triggered.

Table 5.18: Final decision inputs before selecting the baseline PoC metric.

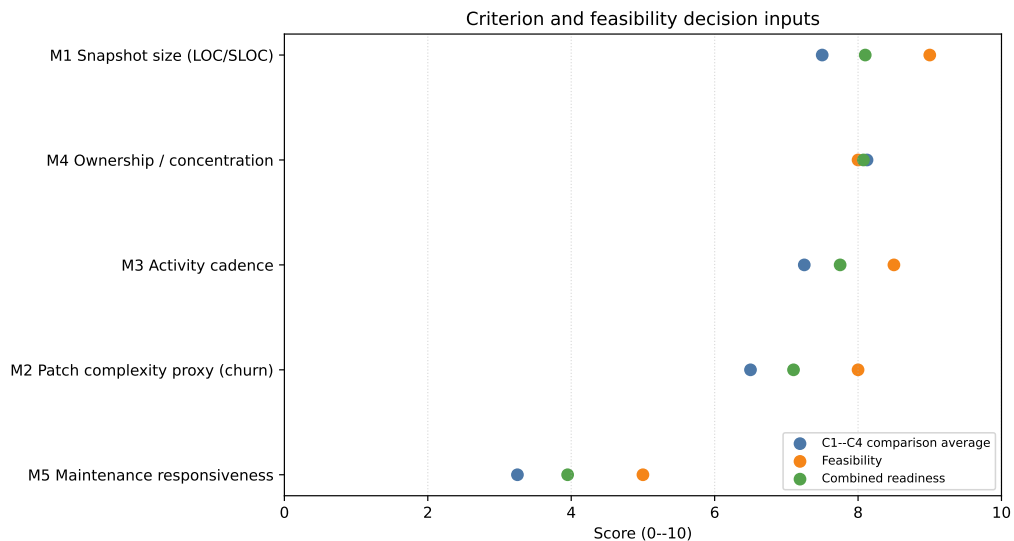


Figure 5.8: Criterion comparison average, feasibility, and combined readiness scores used for the final selection gate. Only C1 is data-derived; C2–C4 are structured author assessments.

The evidence identifies M4 ownership/concentration as the strongest eligible candidate for the final selection gate. This does not mean that M3 and M2 are discarded. Instead, they remain supporting evidence and safeguards: M3 helps interpret activity continuity, and M2 helps check whether ownership changes are backed by actual change effort. M1 remains necessary for context and normalization. M5 is reserved for future work.

Section 5.7 records the final metric-selection rationale, and Phase III instantiates the selected metric in the DESTINO-aligned PoC.

5.7 PHASE II STEP 6: FINAL PoC METRIC SELECTION

The previous sections compared the candidate metrics under C1–C4 and then checked smart-contract feasibility. This section records the final selection decision for the baseline proof-of-concept. The selected metric is not chosen because it is the easiest metric to compute. It is chosen because it has the strongest overall trade-off between reputation meaning, stability, gaming resistance, long-term suitability, and feasibility in the intended DESTINO-aligned hybrid architecture.

5.7.1 SELECTED METRIC

The selected PoC metric is:

***M4 developer churn ownership share:** the share of human-only churn in a frozen release window that is attributed to one canonical developer.*

The concrete signal identifier is `M4_developer_share_churn`. It is computed as:

$$\text{share_churn} = \frac{\text{developer_human_churn}}{\text{window_total_human_churn}}$$

For example, if a release window contains 10,000 human-churn lines and one developer is responsible for 2,500 of those churn lines, that developer’s ownership share for that window is 0.25, or 25%. The selection specifies fixed-point integer storage rather than floating point, a rule implemented in Phase III. Let n be developer churn and d be total human churn. Policy version 1.0.0 defines:

$$\text{share_churn_ppm} = \left\lfloor \frac{n \times 1,000,000 + \lfloor d/2 \rfloor}{d} \right\rfloor, \quad d > 0.$$

This encoding rounds to the nearest ppm with exact ties upward for nonnegative integers. It avoids ambiguous floating-point behavior and is implemented with integer arithmetic in both verifier and chaincode. Independently rounded developer ppm values are not required to sum to exactly 1,000,000.

5.7.2 SELECTION ARTIFACTS

The selection is frozen in dedicated Phase II artifacts so the decision is reproducible and not only written in prose.

5.7. PHASE II STEP 6: FINAL POC METRIC SELECTION

Artifact	Rows	Role
out/poc_metric_selection_all.csv	1	Final selected metric family, concrete signal, scores, and source tables.
out/poc_metric_selection_rationale_all.csv	6	Written reasons supporting the M4 selection.
out/poc_metric_analysis_policy_all.csv	8	Formula, population rule, encoding rule, evidence-hash rule, and supporting checks.
out/poc_metric_record_fields_all.csv	13	Selection-stage compact fields for the PoC metric record.
out/poc_metric_supporting_signals_all.csv	5	Signals retained as context or audit support around the selected metric.
logs/poc_metric_selection_20260708.log	1	Execution log for the metric-selection freeze.

Table 5.19: Final PoC metric-selection artifacts. Paths are relative to the Phase II analysis directory.

5.7.3 WHY M4 IS SELECTED

The selected metric comes from the M4 ownership/concentration family. The decision-input table ranked M4 first among eligible candidates after combining data-derived C1 stability evidence, structured C2–C4 author assessments, and smart-contract feasibility. Its combined readiness score is 8.075. M3 activity cadence remains a strong alternative, with combined readiness 7.750, and M2 churn remains useful supporting evidence, with combined readiness 7.100.

ID	Metric	C1	C2	C3	C4	Feas.	Decision role
M4	Ownership/concentration	8.5	7	9	8	8.0	Selected for the baseline PoC.
M3	Activity cadence	6.0	9	6	8	8.5	Eligible alternative; useful as supporting activity context.
M2	Churn	4.0	8	7	7	8.0	Eligible alternative; useful as supporting effort context.
M1	Snapshot size	10.0	8	5	7	9.0	Not selected because it is context only.
M5	Responsiveness	0.0	6	3	4	5.0	Not selected because current event evidence is absent.

Table 5.20: Selection decision inputs for the final PoC metric.

The main reasons are:

- **Reputation meaning:** M4 measures relative responsibility in a release window, not only activity volume.
- **Gaming-resistance assessment:** M4 received 9/10 under the structured C3 rubric because sustained ownership was judged harder to fake than simple

commit splitting; this is not an empirical attack experiment.

- **Long-term-suitability assessment:** M4 received 8/10 under the structured C4 rubric because relative shares remain interpretable across project sizes; the current dataset does not establish universal long-term behavior.
- **Feasibility:** M4 scored 8.0/10 under the smart-contract feasibility rubric and triggered no exclusion condition.
- **Implementation boundary:** M4 can be computed off-chain and stored as a compact fixed-point record with an evidence hash.

5.7.4 ANALYSIS POLICY

The selected metric must be interpreted with the same policies used in Phase I and Phase II:

- the release window must be one of the frozen `windows_all.csv` intervals and its start commit must be an ancestor of its end commit;
- the developer identity must use the canonicalized author identity policy;
- the primary population must be human-only, with bot rows excluded from the selected metric value;
- the numerator is the developer's human-only churn in the window;
- the denominator is the total human-only churn in the same window;
- the fixed-point value uses the deterministic integer formula above, with scale 1,000,000 and ties rounded upward;
- the evidence packet must retain the analysis policy, source identifiers, numerator, denominator, encoded value, and supporting audit fields.

This policy is important because the value alone is not enough. A value of 250,000 only means 25% ownership if the verifier also knows the exact window, population rule, identity rule, denominator, and encoding scale.

5.7. PHASE II STEP 6: FINAL POC METRIC SELECTION

5.7.5 PoC RECORD FIELDS

At selection time, the baseline PoC record was defined to store compact information and commitments rather than raw repository data. Table 5.21 reproduces the 13-field selection artifact. Phase III refines this concept into the exact 19-field ledger submission and 26-field immutable record schemas, including decimal-string transport for the numerator and denominator before direct `BigInt` parsing.

Field	Type idea	Role
<code>project_id</code>	string/bytes32	Frozen project identifier.
<code>window_id</code>	string/bytes32	Frozen release-window identifier.
<code>developer_id_hash</code>	bytes32	Hash of the canonical developer identifier.
<code>metric_id</code>	string/bytes32	Selection identifier M4_developer_share_churn.
<code>metric_value_ppm</code>	uint32/uint64	Ownership share scaled by 1,000,000.
<code>numerator_churn</code>	uint64	Developer human-only churn.
<code>denominator_churn</code>	uint64	Positive total human-only window churn.
<code>from_commit_hash</code>	bytes20/bytes32	Frozen start commit hash.
<code>to_commit_hash</code>	bytes20/bytes32	Frozen end commit hash.
<code>analysis_policy_hash</code>	bytes32	Hash of extraction, identity, bot, and encoding policy.
<code>evidence_hash</code>	bytes32	Hash of the replayable off-chain evidence packet.
<code>storage_persistence</code>	string/enum	Declared persistence status or reference.
<code>recorded_at</code>	uint64	Timestamp or block-time metadata.

Table 5.21: Selection-stage compact PoC record fields for the selected M4 metric.

5.7.6 SUPPORTING SIGNALS RETAINED

Selecting M4 does not mean the other implemented metrics are discarded. They remain useful as context and audit support:

- M1 endpoint KLOC remains project-size context and normalization support.
- M2 developer churn shows the absolute amount of change behind an ownership share.
- M3 activity cadence helps distinguish sustained activity from one isolated ownership spike.
- M4 top-1/top-3/Gini summaries show whether the whole window is unusually concentrated.
- Bot summaries document whether automation affected the surrounding activity.

The selected PoC metric is therefore a primary record value, not the only evidence used to interpret the record.

5.7.7 DECISION BOUNDARY

This section completes the metric-selection step. The post-hoc ancestry check in Section 5.2 and the deterministic audit in Section 6.2 retain M4 for the baseline PoC without claiming universal superiority. The metric definition must not change unless the selection gate and policy version are reopened. The verifier/chaincode pair submits one replayable M4_DEVELOPER_CHURN_SHARE record from an eligible ancestral window. Its off-chain, local-contract, and network results are reported in Sections 6.3, 6.4, and 6.5.



Proof-of-Concept Design, Implementation, and Validation

6.1 DESTINO PoC INTEGRATION BLUEPRINT

This section records the metric-agnostic integration blueprint used before the Phase II selection decision. Its purpose is to keep Phase I connected to the deployment target while preserving the decision rule: comparison and feasibility scoring happen before implementation. Section 6.2 later instantiates this blueprint for M4 and Hyperledger Fabric.

6.1.1 DESIGN OBJECTIVE AND BOUNDARY

The PoC is required to support evidence-based developer reputation records that are traceable to hash-pinned software artifacts. The contract layer does not compute repository metrics directly. Repository traversal, Git history analysis, duration normalization, bot handling, and ownership calculations remain off-chain because they require file-system access, Git tooling, and potentially unbounded iteration. The contract stores the result and the commitments needed to audit it.

The boundary is:

- **Off-chain:** artifact retrieval, hash verification, metric extraction, cleaning, normalization, and report generation;

6.1. DESTINO POC INTEGRATION BLUEPRINT

- **On-chain:** submission registration, compact metric-record storage, evidence-hash storage, permission checks, and events;
- **Not in Phase I:** final metric selection, contract implementation, and M5 maintenance-responsiveness implementation.

6.1.2 METRIC-AGNOSTIC RECORD SHAPE

The initial contract interface remains metric-agnostic until Phase II selects a metric. A minimal record shape is shown in Table 6.1; field names are conceptual placeholders rather than final chaincode declarations.

Record	Core fields	Role
ProjectSubmission	projectId, developerId, versionId, archiveLink, archiveHash, tags, storagePersistence, submittedAt	Registers the artifact fact/provenance claim that later metric evidence depends on.
MetricRecord	projectId, versionId, metricName, metricValue, evidenceHash, analysisPolicy, recordedAt	Stores the compact selected metric result and binds it to an auditable off-chain report.

Table 6.1: Metric-agnostic record shape used during DESTINO PoC planning.

The `analysisPolicy` field is included because the Phase I evidence makes population and sensitivity rules explicit: human-only M3/M4 outputs, short-window flags, and bot-sensitivity summaries are not incidental cleaning details. They are part of the evidence interpretation that must accompany any instantiated metric record.

6.1.3 TRANSACTION AND REPLAY FLOW

The intended end-to-end flow is:

1. a developer registers signed project/archive metadata and an archive hash;
2. an off-chain verifier retrieves the artifact and checks hash equality;
3. the selected Phase II metric is computed using the pinned script version and declared analysis policy;

4. a compact metric value and evidence-report hash are recorded on-chain;
5. an event is emitted so an indexing/search component can expose the new data point;
6. auditors can replay the same computation from the artifact hash, script version, window definition, and evidence report.

This design preserves the core DESTINO requirement: trust is not based on a private metric calculation alone, but on a replayable chain of artifact commitments, deterministic scripts, and compact auditable records.

6.1.4 HOW PHASE I OUTPUTS SUPPORT THE BLUEPRINT

The Phase I outputs support the subsequent integration:

- frozen projects, versions, and windows provide stable input identifiers;
- clean and human-only metric tables provide candidate values that can be reduced to compact records;
- duration flags and bot summaries document analysis policies that affect interpretation;
- logs and reproducibility checkpoints provide the replay trail for third-party verification;
- the feasibility rubric defines the gate that prevents an empirically attractive metric unsuitable for bounded contract execution from becoming the PoC.

This blueprint establishes the planning and evidence boundary. Phase II subsequently selects M4, and Section 6.2 freezes the concrete calculation, schemas, platform profile, and trust model before implementation.

6.2 PHASE III: DETERMINISTIC M4 AND HYBRID FABRIC PoC DESIGN

Phase III begins by freezing the selected signal as an implementation contract before writing verifier or chaincode code. This avoids a common reproducibility

6.2. PHASE III: DETERMINISTIC M4 AND HYBRID FABRIC POC DESIGN

failure in which the metric formula, rounding, evidence fields, and trust assumptions change during implementation. The phase adds only new artifacts under `Data, Outputs, Logs, Scripts/Phase3PoC/`; the Phase I, Phase II, and post-hoc inputs remain unchanged.

6.2.1 VERSIONED METRIC POLICY AND ARITHMETIC AUDIT

The implementation identifier is `M4_DEVELOPER_CHURN_SHARE`. The associated policy is `M4_DEVELOPER_CHURN_SHARE_POLICY`, version `1.0.0`. Its canonical policy SHA-256 is:

`4e81843340041da45a13ebb742a0c5fdfebd6c8f5e64a7d6629f58990b287a4`

The policy freezes the original extraction semantics: non-merge git log `--numstat` history, author email (`%ae`) as the canonical identity source, case-insensitive regex bot exclusion, all changed paths without a source-language filter, and binary changes contributing zero textual churn. The PoC additionally requires linear Git ancestry and uses the exact integer ppm formula in Section 5.7.1.

An independent validator reconstructs developer churn, denominators, individual shares, top-one share, and Gini directly from the human developer table. It then checks the frozen ownership tables and the Phase II C1 source values.

Validation quantity	Result
Developer rows checked	1,063
Windows checked	70
Ancestral/eligible PoC windows	58
Developer rows in eligible windows	848
Developer calculation failures	0
Window-total failures	0
Ownership-summary failures	0
Phase II C1 source-value failures	0
Zero-denominator windows	0

Table 6.2: Deterministic M4 calculation audit.

The maximum discrepancy between exact rational values and the frozen six-decimal CSVs is 4.98168×10^{-7} for individual shares, 4.92063×10^{-7} for top-one

share, and 4.95298×10^{-7} for Gini. These values are below the expected half-unit at six decimal places. A second implementation checked all 1,063 ppm values, and an alternative pairwise Gini formula checked all 70 windows with no mismatch. Six automated policy tests include boundary cases, invalid inputs, a fixed fixture, and exhaustive comparison with decimal half-up rounding for denominators from 1 to 500.

6.2.2 FABRIC PLATFORM PROFILE

The baseline freezes Hyperledger Fabric 2.5.15 from the 2.5 LTS line. The official Fabric test network is Docker-Compose based and contains two peer organizations and an ordering organization; its documentation explicitly positions it as an education and testing environment rather than a production template [5, 4]. JavaScript and TypeScript chaincode are supported by the documented deployment workflow [3]. JavaScript is selected because the local environment already provides Node and Docker, while no Go toolchain is installed.

The frozen network profile uses one peer for `Org1MSP`, one peer for `Org2MSP`, a single-node Raft orderer, channel `mychannel`, default `GoLevelDB` state, and JavaScript chaincode `m4evidence`. Fabric Node libraries are pinned at 2.5.8. Both organizations endorse writes, while `Org1MSP` is the authorized evidence submitter in the baseline scenario.

6.2.3 OFF-CHAIN/ON-CHAIN BOUNDARY

Layer		Responsibilities	Explicit non-responsibilities
Off-chain verifier		Ancestry validation, Git extraction, email normalization, bot policy, integer M4 calculation, canonical manifest creation, and policy/artifact/manifest hashing.	Ledger ordering, endorsement, or durable consensus.
Fabric code	chain-	Submitter authorization, format/range validation, integer ppm recomputation, duplicate rejection, compact state/index writes, and event emission.	Repository access, ancestry replay, bot truth, semantic code quality, or external storage availability.
Independent auditor		Retrieve the committed artifact, verify hashes, rerun the versioned verifier, and compare the result with the ledger.	Changing the policy while claiming to reproduce the same record.

Table 6.3: Hybrid PoC computation and trust boundary.

This boundary is central to the thesis claim. Fabric consensus establishes which endorsed record was committed; it does not make the off-chain Git analysis true. Evidence credibility comes from pinned inputs, deterministic open tooling, explicit policy versioning, hash commitments, authenticated submission, and independent replay.

6.2.4 EVIDENCE AND LEDGER SCHEMAS

Three JSON schemas separate deterministic evidence from transaction-derived metadata.

Schema	Purpose
M4_EVIDENCE_MANIFEST	Canonical replay description containing policy, repository, ancestral window, pseudonymous subject, extraction totals, integer calculation, evidence-artifact commitment, and verifier version/source commitment.
M4_LEDGER_SUBMISSION	Compact client payload containing the fields required for chaincode validation. Numerator and denominator are decimal strings so JavaScript converts them directly to BigInt without precision loss.
M4_LEDGER_RECORD	Validated submission plus deterministic record ID, Fabric submitter MSP and identity hash, transaction ID, transaction time, and immutable ACTIVE status.

Table 6.4: Versioned evidence and ledger schemas for the baseline PoC.

Policy and manifest hashes use UTF-8 JSON with lexicographically sorted object keys, no insignificant whitespace, no non-finite numbers, and no trailing newline in the hashed bytes. Dynamic values such as Fabric transaction ID, submitter identity, and transaction time are excluded from the evidence manifest, so identical evidence produces identical manifest bytes. The manifest does not contain its own hash; that SHA-256 commitment is placed in the ledger submission.

The chaincode derives a composite primary key from project, window, developer hash, metric, and policy version. It rejects an existing key rather than overwriting evidence. Secondary composite keys support project and developer queries without requiring CouchDB rich queries. A successful write emits `M4MetricEvidenceSubmitted`.

6.2.5 TRUST MODEL AND RESIDUAL THREATS

The baseline authorizes `Org1MSP` as verifier/submitter and requires endorsement from both organizational peers. `Org2` therefore participates in ledger validation, but endorsement alone does not prove the numerator and denominator. An auditor must retrieve bytes matching the artifact hash and replay the matching policy/verifier version.

Hash commitments detect changed bytes but do not guarantee availability. Canonical email hashing is pseudonymization rather than anonymization and

6.3. PHASE III: OFF-CHAIN VERIFIER AND EVIDENCE REPLAY

does not prove real-world control of the email. The frozen bot regex remains heuristic. Churn can be inflated through meaningless edits, and the ledger makes such evidence replayable rather than automatically determining its semantic value. These limitations keep the result correctly framed as one auditable evidence signal, not a self-sufficient reputation verdict.

6.2.6 BASELINE INTEGRATION FIXTURE

The first integration fixture is the ancestral `urllib3_W03` window from tag 2.4.0 to 2.5.0. It belongs to the primary duration set, contains nine human developers, and exercises bot exclusion through one Dependabot identity. Human churn is 1,058 lines. The selected top contributor has numerator 325, exact share $325/1058 = 0.307183364839\dots$, and deterministic ledger value 307,183 ppm. The pinned fixture includes both 40-character commit hashes, the pseudonymous developer hash, policy hash, bot context, supporting Gini, and expected rounding result.

The design validator checks all three schemas, local references, required fields, shared ledger constraints, policy constants, and fixture values. It reports zero failures and emits canonical hashes for the policy, design, and schemas. This completes the design gate used by the off-chain implementation in Section 6.3, the chaincode implementation in Section 6.4, and the network evaluation in Section 6.5; all were implemented against the same fixed contracts rather than evolving prose.

6.3 PHASE III: OFF-CHAIN VERIFIER AND EVIDENCE REPLAY

The first implemented PoC boundary is the off-chain evidence path. It does not read a selected row from the frozen ownership CSV. Instead, it resolves the pinned Git tags, checks commit ancestry, traverses the release range, applies the frozen identity and bot policies, recomputes M4 using integer arithmetic, creates the evidence commitments, and emits the ledger submission defined by the frozen schema.

6.3.1 STANDALONE GIT EVIDENCE ARTIFACT

The evidence builder creates a Git bundle version 2 containing both annotated tag refs, 2.4.0 and 2.5.0. No prerequisite object is declared, so the artifact can populate a fresh repository without access to the original working copy. To test build determinism, the builder creates two bundles with single-thread packing and rejects the result unless their raw-byte SHA-256 values are identical.

The resulting artifact is 7,206,262 bytes. Its SHA-256 is:

```
a41205eff782ce4af65f345e0afb97f16fd8677af215c7bdd06f460ce9602272
```

The bundle remains a local PoC artifact under the declared LOCAL_POC persistence class. The hash detects changed bytes but does not guarantee long-term availability.

6.3.2 DIRECT VERIFIER RESULT

The Python verifier follows the frozen Phase I extraction semantics, including non-merge `git log --numstat`, lowercased and trimmed author email, most-frequent display-name selection with a lexical tie-break, regex bot exclusion, and zero textual churn for binary-file changes. It additionally fails closed on malformed Git output, tag/hash disagreement, failed ancestry, non-positive duration or denominator, fixture disagreement, schema error, and cross-contract mismatch.

6.3. PHASE III: OFF-CHAIN VERIFIER AND EVIDENCE REPLAY

Verified quantity	Value
Tagger-time duration	5,957,933 seconds
Non-merge commits	16
Human developers / commits	9 / 14
Human added / deleted lines	703 / 355
Human churn denominator	1,058
Excluded bot identities / commits	1 / 2
Excluded bot churn	18
Binary-file changes	0
Selected developer churn numerator	325
Deterministic M4 value	307,183 ppm
Schema / fixture / cross-contract failures	0 / 0 / 0

Table 6.5: Direct Git extraction and M4 verifier result for `urllib3_W03`.

The verifier writes the evidence manifest and ledger submission as UTF-8 canonical JSON without a trailing newline. The manifest contains no local path or generation time. The verifier source, manifest, and compact submission commitments are:

Committed object	SHA-256
Verifier source	b4e8171dfe228aec96c8e67de526f037af253bc85bd502cc6bc5c96991aedb19
Evidence manifest	df9e35adde01937b12da29eb21a3135268705d84168f9f1159a086b769d583ec
Ledger submission	077e65f900aedc0f066cc01725a62f721c314a0932cdcd3b6f722c0c17a14fbe

Table 6.6: Off-chain implementation commitments.

6.3.3 FRESH-BUNDLE REPLAY AND INDEPENDENT TRAVERSAL

The replay check initializes an empty bare repository and fetches only the two tag refs from the bundle. Running the primary verifier in that repository reproduces four files byte-for-byte: the manifest, ledger submission, developer audit, and verification summary. Their hashes are unchanged across repeated complete pipeline runs.

A second extraction route checks for a shared-parser error. It enumerates the 16 non-merge commits with `git rev-list`, reads author metadata per commit, and accumulates each commit's changes with `git diff-tree --numstat`. This

route does not use the verifier’s `git log --numstat` parser. It independently matches every commit, developer, bot, added, deleted, churn, binary, numerator, denominator, and ppm field committed in the manifest.

Thirteen automated tests pass. They cover the exhaustive ppm test from Section 6.2.1, canonical serialization, identity normalization, bot and binary handling, generated-schema validation, the artifact–manifest–submission hash chain, and zero-failure summaries. Negative tests reject both an additional submission field and a one-ppm arithmetic mutation. Repeated execution checks nine generated files and logs with zero hash differences.

6.3.4 INTERPRETATION AND OFF-CHAIN BOUNDARY

This step establishes deterministic extraction, calculation, serialization, commitment, and local replay for one ancestral fixture. It does not prove control of an author email, perfect bot identification, semantic value of churn, permanent bundle availability, or generalization beyond the evaluated windows. Section 6.4 tests authorization, duplicate prevention, state, history, queries, and events at the contract-unit boundary; Section 6.5 then evaluates endorsement, ordering, commit observation, authenticated rejection, and Gateway delivery. The bundle necessarily retains the repository’s original author metadata; the pseudonymous developer hash limits the compact ledger subject field but does not anonymize the off-chain evidence artifact.

6.4 PHASE III: CHAINCODE IMPLEMENTATION AND LOCAL VALIDATION

The second implemented PoC boundary is the JavaScript `m4evidence` contract. It consumes the canonical submission produced in Section 6.3; it does not accept repository paths, raw commits, or client-supplied Fabric metadata. The implementation pins `fabric-contract-api` and `fabric-shim` at version 2.5.8 and exposes exactly six public transactions: `submit`, `exists`, `read`, `history`, `query by project`, and `query by developer`.

6.4.1 VALIDATED STATE TRANSITION

`SubmitMetricEvidence` first permits only `Org1MSP` under the baseline policy. It requires the exact 19-field submission shape and validates all schema, metric, policy, identifier, Git-hash, SHA-256, persistence-class, and range constraints. The numerator and denominator arrive as canonical decimal strings and are converted directly to `BigInt`. The contract requires $0 \leq n \leq d$, $d > 0$, and independently recomputes

$$\left\lfloor \frac{n \times 1,000,000 + \lfloor d/2 \rfloor}{d} \right\rfloor.$$

Thus, no numerator or denominator passes through JavaScript floating point. A mismatching submitted ppm value is rejected. The primary composite key contains project, window, developer hash, metric, and policy version; an existing key is rejected instead of overwritten. On success, the contract stores the exact 26-field record required by the frozen schema, writes project and developer secondary indexes, and emits `M4MetricEvidenceSubmitted`. The record identifier and submitter identifier are domain-separated SHA-256 values. The submitter MSP, client identity, transaction ID, and transaction timestamp come from the Fabric context rather than the client payload.

6.4.2 LOCAL TEST RESULT

The contract was executed against deterministic mocks implementing the Fabric state, composite-key, history, event, transaction, and client-identity interfaces. The canonical `urllib3_W03` ledger submission was used as the positive fixture. Table 6.7 summarizes the reproducible result.

Validation quantity	Result
Public contract transactions	6
Ledger-record required fields	26
Node tests passed / total	34 / 34
Failed / skipped tests	0 / 0
Production dependency audit findings	0
Packaged source files	3
Unpacked chaincode package size	15,053 bytes

Table 6.7: Local m4evidence chaincode validation result.

The tests check exact submission-to-record mappings; deterministic record and submitter hashes; state, index, and event writes; read, existence, history, and both index queries; immutable duplicate rejection; and `Org2MSP` rejection. Negative cases cover malformed JSON, missing or additional fields, invalid constants and hashes, non-canonical decimal strings, zero denominator, numerator above denominator, non-integer ppm, and a one-ppm mutation. Arithmetic boundary cases cover zero, one, one half, half-ppm upward rounding, and integers above `Number.MAX_SAFE_INTEGER`. The resulting record ID for the fixture is:

72ca9db25cfc77fdb95592a2b1b2debbf9bbf006a16a419dd97a31c7b57309

The domain-separated chaincode source-set commitment and the dependency-lock commitment are:

Committed object	SHA-256
Chaincode source set	9f41f699a0cd0c8db239c8ccdcf8b9ac3113ec65b26a9872b4015173a5b0d0be
Dependency lock file	f16d29bd8ce1295b79a68313ad359c66bbb25f45cca2d13218a715ec8cfb20c0

Table 6.8: Chaincode implementation commitments.

6.4.3 INTERPRETATION AND NETWORK BOUNDARY

These results establish that the implemented contract logic satisfies the frozen arithmetic and state-transition rules under the tested Fabric interfaces. They do not by themselves demonstrate peer agreement, two-organization endorsement, ordering, commit status, Gateway event delivery, or persistence in a running Fabric channel; those properties are evaluated with the unchanged

6.5. PHASE III: FABRIC NETWORK DEPLOYMENT AND INTEGRATION VALIDATION

package in Section 6.5. A syntactically valid but false evidence hash remains outside the chaincode’s knowledge: an auditor must retrieve the artifact and replay the off-chain verifier.

6.5 PHASE III: FABRIC NETWORK DEPLOYMENT AND INTEGRATION VALIDATION

The final implementation boundary deploys the unchanged `m4evidence` contract from Section 6.4 and submits the canonical `urllib3_w03` evidence from Section 6.3 through an actual Fabric Gateway client. This step tests the properties that local contract mocks cannot establish: organizational approval and endorsement, ordering and commit status, authenticated identity handling, peer-visible state, channel history, and Gateway event delivery.

6.5.1 FROZEN ENVIRONMENT AND DEPLOYMENT

The run used Fabric v2.5.15 (commit 83c7930) from the Fabric 2.5 LTS line, Fabric CA binaries v1.5.17, Docker Engine 29.4.3, and Docker Compose 5.1.3. The official `fabric-samples` test network was detached at commit 65592350d7d7c51b02c8a4d89383d4bbdcc45725. Because the installer did not expose a matching `fabric-samples v2.5.15` tag, recording this commit avoids treating a moving sample branch as a frozen input. Fabric identities were generated locally with cryptogen; Fabric CA was installed as part of the frozen tool set but no CA service was used. This wording distinguishes configured X.509 MSP identities from CA enrollment.

The topology contains `peer0.org1.example.com`, `peer0.org2.example.com`, and one Raft orderer. Both peers joined `mychannel`, using `GoLevelDB`. Four byte-checked runtime files were staged from the locally tested source and packaged for deployment. Table 6.9 records the committed definition.

Deployment property	Measured value
Chaincode name / version / sequence	m4evidence / 1.0 / 1
Package size / source files	13,275 bytes / 4
Package SHA-256	5e767ab94efbb4a63b8958f44249b0c0c2d7cede71738593c813282f091fabb7
Organization approvals	Org1MSP=true; Org2MSP=true
Endorsement policy	AND('Org1MSP.peer', 'Org2MSP.peer')
Gateway / gRPC versions	1.10.1 / 1.14.4
Production dependency findings	0

Table 6.9: Committed chaincode definition and client environment.

The validator queried the committed definition on both peers and decoded its binary validation parameter. It requires a threshold of two with the Org1MSP PEER and Org2MSP PEER principals. This check is stronger than matching only a displayed policy string. The package was installed on both peers with identifier `m4evidence_1.0:5e767ab94efbb4a63b8958f44249b0c0c2d7cede71738593c813282f091fabb7`.

6.5.2 AUTHENTICATED POSITIVE AND NEGATIVE PATHS

The Gateway client used the generated User1 certificate and private key for each organization. The accepted proposal used the Org1 identity permitted by chaincode and explicitly requested endorsements from both organizations. The 17 integration checks all passed; Table 6.10 groups them by purpose.

6.5. PHASE III: FABRIC NETWORK DEPLOYMENT AND INTEGRATION VALIDATION

Group	Checks	Passed
Precondition and rejection	independent record ID, absent initial state, malformed JSON, one-ppm mutation, unauthorized Org2 submitter, and no state from rejected proposals	6/6
Commit and event	valid Org1 commit, exact 26-field proposed record, Gateway event reception, and exact event payload	4/4
Ledger observation	state existence, identical reads from both peers, project query, developer query, and exact history	5/5
Immutability	duplicate rejection and preservation of original state	2/2
Total		17/17

Table 6.10: Authenticated Fabric Gateway integration checks.

Malformed JSON, the one-ppm mutation, and the Org2 submission failed during endorsement and created no state. After the valid commit, the project and developer indexes returned the same record, both peers returned byte-equivalent parsed state, and history contained the single expected update. A second valid submission was rejected and did not alter that state.

6.5.3 COMMITTED TRANSACTION AND INDEPENDENT VALIDATION

The accepted record retained the deterministic identifier already derived in the local tests:

```
72ca9db25cfcd77fdba95592a2b1b2debbf9bbf006a16a419dd97a31c7b57309.
```

Its transaction identifier is:

```
960e5630c0427ca7f74c87930b4611646c9e124969efb31b6514dca5c95be031.
```

The transaction was committed in block 6 at 2026-07-26T13:21:13.565Z; the channel height was seven after the run. The event name was M4MetricEvidenceSubmitted, and the canonical 26-field record SHA-256 was 0f6840a59706211533ac4e84987a26baf89dcccce222f8b61ef22bd443ebe3d29. An independent Python validator retrieved block 6 through each peer's local QSCC service, found the transaction, and confirmed validation code 0 (VALID). Thus the result is not inferred only from a successful Gateway return.

The independent validation passed all 47 environment, packaging, lifecycle, policy, Gateway-result, dependency, block, and artifact-integrity checks. As a separate replay negative case, it changed one byte in a temporary copy of the Git bundle. The copy's SHA-256 changed from `a41205eff782ce4af65f345e0afb97f16fd8677af215c7bdd06f460ce9602272` to `debeb80119a125f617cfdc5ec33537460ab07625a49592711d56601e2f203fdf`, and independent replay rejected it. The canonical bundle remained unchanged.

6.5.4 INTERPRETATION AND LIMITS

The network run confirms the implemented mechanism for one pinned evidence fixture: authenticated authorization, exact arithmetic validation, two-organization endorsement, immutable commit, consistent peer reads, queries, history, event delivery, and rejection paths work together on the frozen local topology. It does not show that the off-chain numerator and denominator are semantically true merely because Fabric committed them; that claim remains auditable through the artifact, policy, verifier, and hashes.

The evaluation uses a single-host educational network, generated test identities, one orderer, one peer per organization, one fixture, and no load or fault experiment. It therefore supports a bounded proof-of-concept implementation claim, not production claims about throughput, scale, availability, certificate governance, privacy, or adversarial consensus. Exact commands, container-image digests, source/output commitments, and machine-readable results are retained in the Phase III `FABRIC_NETWORK_REPORT.md`, `fabric_runtime/README.md`, and `out/evidence`.



Discussion and Conclusions

7.1 THREATS TO VALIDITY

7.1.1 CONSTRUCT VALIDITY

M2 uses churn as a proxy for change magnitude, which does not directly encode semantic complexity, defect risk, or architectural importance. M3 cadence features are sensitive to timestamp semantics; commit-date and tag-date boundaries are therefore treated as robustness concerns rather than ignored. Automated accounts can inflate commit-frequency and ownership/cadence interpretations, so M3 and M4 use human-only outputs as the primary population and keep all-account outputs for sensitivity checks. To reduce construct ambiguity, the study keeps raw fields such as added lines, deleted lines, commits, and activity days alongside derived indicators.

7.1.2 INTERNAL VALIDITY

Potential extraction errors were mitigated through reproducible tag-chronological windows, immutable commit hashes, explicit cleaning rules, and arithmetic checks such as $\text{churn} = \text{added} + \text{deleted}$. Developer identity is canonicalized by email during extraction and cleaning, window totals are recomputed from clean developer rows, and ownership summaries are regenerated from those same clean rows. Bot-sensitivity outputs also validate that all-account totals remain consistent before deriving human-only tables.

7.1. THREATS TO VALIDITY

The original tag-date rule is deterministic but does not guarantee linear Git ancestry. A post-hoc audit found 12 non-ancestral pairs containing 41.77% of human commits and 71.55% of human churn. The original analysis is retained as an exploratory baseline; the divergent pairs are excluded from the ancestry-only sensitivity analysis and from PoC eligibility. JUnit has no retained window in that subset, and only eight projects have enough retained primary windows for the primary sensitivity summary. This reduced sample is reported explicitly rather than treated as a corrected ten-project result.

Email normalization does not resolve one person using multiple addresses and does not prove control of an address. The developer hash used by the PoC is therefore a pseudonymous technical identifier, not a verified civil identity.

7.1.3 EXTERNAL VALIDITY

The final Phase I dataset is broader than the pilot because it covers ten repositories across Python, Ruby, C/C++, Go, Java/Kotlin, JavaScript/TypeScript, and Rust ecosystems. This improves transferability compared with the four-repository pilot, but it is still a curated open-source sample rather than a statistically representative sample of all software projects. Conclusions are therefore reported as evidence from this selected dataset, not as universal claims about every repository or ecosystem. The baseline Fabric network is an educational two-organization test network and cannot support production-level claims about governance, availability, scale, or Byzantine resilience. The 34 chaincode tests establish local contract logic with deterministic Fabric-interface mocks. The subsequent live run adds evidence for two-peer endorsement, ordering, Gateway delivery, and persistent channel behavior on one single-host topology and one fixture; it does not establish performance, fault tolerance, or production generalizability.

7.1.4 CONCLUSION VALIDITY

The C1 stability results are calculated from repository evidence. C2 interpretability, C3 gaming resistance, and most C4 long-term-suitability scores are structured author assessments encoded in reproducible rubrics; replaying a script reproduces those judgments but does not make them empirical measurements or independent ratings. The combined readiness score and its 60/40 weighting are therefore decision aids rather than inferential statistics.

The supported conclusion is limited: under the thesis evaluation framework and ancestry-only sensitivity check, M4 is a suitable implemented signal for the baseline PoC. The study does not establish that M4 is universally optimal, that it directly measures developer quality, or that gaming resistance is empirically proven. The selected individual developer share is related to, but not identical with, the aggregate Gini and top-one signals used in the window-level C1 score; low contributor overlap further limits longitudinal individual-level interpretation.

Finally, the test-network run confirms that Fabric chaincode can authenticate the configured submitter, validate integer arithmetic, prevent duplicate state, and bind hash commitments. It cannot independently traverse Git, establish ancestry, validate bot classification, guarantee external artifact availability, or judge semantic code quality. These claims remain dependent on versioned off-chain verification and independent replay. The successful transaction therefore validates the implemented mechanism, not the semantic truth or universal usefulness of M4.

7.2 DISCUSSION AND FINAL CONCLUSION

This thesis asked which repository-derived indicators are sufficiently stable, interpretable, resistant to manipulation, and technically feasible to support a developer-reputation proof-of-concept. No single metric dominates every criterion. The evidence instead supports a layered result: M1 is the strongest size/context signal, M2 records change magnitude, M3 provides the clearest activity-continuity signal, and M4 provides the best overall trade-off for the baseline reputation-oriented PoC. Phase III then demonstrates how the selected M4 value can be made deterministic, replayable, and auditable across the off-chain/on-chain boundary.

7.2.1 SYNTHESIS OF THE EVIDENCE

The study combines four forms of evidence that should not be treated as interchangeable. First, Phase I provides empirical repository measurements from ten curated projects, 80 frozen tags, and 70 tag-chronological windows. Second, C1 calculates stability from those observations. Third, C2–C4 and the feasibility rubric encode transparent structured author assessments; their reproducibility

7.2. DISCUSSION AND FINAL CONCLUSION

preserves the stated judgments but does not convert them into measured facts or independent ratings. Fourth, Phase III supplies implementation evidence through deterministic replay, contract tests, authenticated network transactions, and independent inspection of committed channel data.

This separation is necessary for interpreting the final decision. The 60/40 readiness weighting orders candidates within the declared thesis framework; it is not an inferential model. Likewise, a valid Fabric transaction demonstrates that the implemented authorization, arithmetic, endorsement, and state-transition rules operated as specified. It does not establish that churn is semantic quality or that an off-chain claim becomes true merely because it is committed to a ledger.

RQ	Most relevant evidence	Supported answer
RQ1	Window CV, adjacent-window rank correlation, contributor overlap, short-window sensitivity, and ancestry-only sensitivity	M1 is the most stable context signal. Among behavior-oriented window signals, M4 concentration is least volatile; M3 has the strongest rank continuity among recurring developers.
RQ2	C2 and C3 structured rubrics, bot sensitivity, identity policy, and retained supporting fields	M3 is easiest to explain; M4 has the strongest assessed gaming-resistance profile, but neither is a direct quality measure and the gaming result is not an attack experiment.
RQ3	C1–C4 comparison, feasibility gate, deterministic replay, local contract tests, and live Fabric validation	M4 is the strongest eligible baseline PoC candidate under the declared framework and is implementable as replayable off-chain evidence plus a compact, validated ledger commitment.

Table 7.1: Thesis-level answers and their evidence boundaries.

7.2.2 ANSWER TO RQ1: STABILITY AND COMPARABILITY

RQ1 asks which indicators are stable across versions and comparable across projects. At window level, M1 endpoint KLOC is the most stable signal, with primary median CV 0.014, because project size changes gradually between releases. This makes M1 valuable for context and normalization, but not as standalone developer-reputation evidence. Among behavior-oriented signals, M4 is the more stable family: primary median CV is 0.236 for top-one churn share

and 0.329 for churn Gini, compared with 0.608 for M3 commits/day and 0.890 for M2 churn/KLOC/day.

The post-hoc ancestry result materially qualifies the original analysis. Twelve of the 70 tag-chronological windows are not linear ancestor-to-descendant intervals and contain 41.77% of human commits and 71.55% of human churn. Excluding them leaves 58 eligible windows. The qualitative behavioral ordering is retained: ancestry-only primary median CV is 0.247 for M4 top-one share, 0.351 for M4 Gini, 0.631 for M3, and 0.890 for M2. This supports retaining M4 for the engineering PoC, but the reduced eight-project primary summary cannot be presented as a corrected ten-project population.

Developer-rank stability adds a different perspective. Among developers appearing in adjacent windows, M3 commits/day has the strongest primary median Spearman correlation (0.836), while M2 churn/day and M4 churn share both have 0.608. M2 and M4 produce the same within-window ordering because dividing every developer's churn by the same window denominator changes the unit, not the rank. Moreover, the median common-developer share is only 0.172. The supported RQ1 answer is therefore conditional: M4 concentration is the least volatile behavior-oriented window signal, whereas M3 is the stronger continuity signal for the small recurring-developer subset. No result justifies treating one developer score as universally stable across changing contributor populations.

7.2.3 ANSWER TO RQ2: INTERPRETATION AND MANIPULATION

RQ2 asks which indicators are understandable and robust against manipulation. Under C2, M3 receives 9/10 and is the clearest implemented reputation-adjacent signal. M1 and M2 each receive 8/10, while M4 receives 7/10 because ownership shares, concentration, canonical identity, and the denominator must all be explained together. Interpretability alone therefore would favor M3, not M4.

The structured C3 assessment reverses that ordering: M4 receives 9/10, M2 7/10, M3 6/10, and M1 5/10. Sustained relative ownership was judged more difficult to manufacture than commit counts, which can be increased through splitting or automation. This is a reasoned rubric result, not empirical proof of gaming resistance. M4 can still be distorted through meaningless high-churn edits, dominance by one unusually large change, bot misclassification, email

7.2. DISCUSSION AND FINAL CONCLUSION

aliases, or spoofed identity metadata. The implementation improves detectability by retaining numerator, denominator, bot context, window commitments, and replayable Git evidence; it does not automatically decide whether the edits are valuable.

The supported RQ2 answer is consequently a trade-off. M3 is the easiest to communicate but needs commit-splitting and bot safeguards. M4 is more demanding to explain but offers the strongest assessed reputation meaning and manipulation cost when its identity and evidence rules are explicit. M1, M2, and M3 should remain supporting context rather than being discarded: size frames the project, churn exposes the absolute work behind a share, and cadence distinguishes sustained activity from an isolated ownership spike.

7.2.4 ANSWER TO RQ3: PoC SELECTION AND TECHNICAL FEASIBILITY

RQ3 asks which indicator is the strongest PoC candidate under implementation constraints. M4 receives a C1–C4 comparison average of 8.125 and feasibility 16/20 (normalized to 8.0). Its weighted readiness score is 8.075, above eligible alternatives M3 at 7.750 and M2 at 7.100. M1’s numerical readiness is high, but it is ineligible as a standalone reputation metric because it describes project context. M5 is excluded because the current dataset contains no issue or pull-request event stream from which responsiveness can be replayed.

The implementation results support the feasibility part of this answer. Exact integer arithmetic independently checks all 1,063 developer rows and 70 window summaries with zero failures. For the eligible `urllib3_W03` fixture, independent Git extraction reproduces numerator 325, denominator 1,058, and 307,183 ppm. A standalone bundle supports fresh-repository replay; thirteen off-chain tests and an independent Git traversal confirm the extraction and commitment chain. The unchanged JavaScript contract then passes 34 local tests covering authorization, schema constraints, `BigInt` arithmetic, immutability, indexes, history, and events.

The live network closes the remaining implementation gap. The package was installed on both peers and approved by both organizations, and the committed validation parameter was independently decoded as requiring both organizational peers. All 17 authenticated Gateway checks and all 47 independent network checks pass. The accepted Org1 transaction is committed with validation code 0 in block 6, is read identically through both peers, appears in both indexes

and history, and emits the expected event. Malformed JSON, a one-ppm mutation, an Org2 submission, a duplicate, and a temporary one-byte artifact modification are rejected through their respective on-chain or replay boundaries.

The supported RQ3 answer is that M4 is feasible in a hybrid architecture, not that Git analysis should occur in chaincode. Git traversal, ancestry, bot classification, identity normalization, and aggregation remain off-chain; Fabric authenticates the configured submitter, recomputes the compact arithmetic, requires the declared endorsements, prevents overwrite, and binds the result to replayable evidence through hashes.

7.2.5 IMPLICATIONS FOR A DESTINO-ALIGNED REPUTATION SYSTEM

The main architectural implication is that a reputation system should preserve evidence layers instead of reducing activity to an opaque score. A compact ledger record can identify the project, window, pseudonymous subject, policy version, exact integer value, numerator, denominator, and evidence commitments. An auditor can then retrieve the matching artifact and reapply the same policy. This design makes disagreement inspectable: a party can challenge the artifact, extraction, identity rule, or interpretation without changing the already committed record.

Policy versioning is equally important. Changing bot rules, identity linkage, release-window eligibility, or rounding creates a different measurement contract and should produce a new policy version rather than silently rewriting prior evidence. The ledger's role is therefore provenance, authorization, ordering, and immutability under configured governance. It is not an oracle for developer identity, semantic code quality, or the long-term availability of external artifacts.

7.2.6 LIMITATIONS AND FUTURE WORK

The limitations in Section 7.1 bound the conclusion. The ten repositories are curated rather than a statistical sample of all software ecosystems. The ancestry-only subset removes all JUnit windows and leaves only eight projects able to contribute to its primary stability summary. Email canonicalization is pseudonymous and does not resolve aliases or prove real-world identity. C2–C4 are structured author assessments without independent raters, and the 60/40 weighting is a design choice. Finally, the Fabric evaluation uses one fixture, generated test

7.2. DISCUSSION AND FINAL CONCLUSION

identities, one peer per organization, one orderer, and no performance or fault experiment.

The most valuable extensions would be to replay multiple eligible windows and developers, add issue and pull-request evidence for M5, evaluate explicit identity-linking governance, recruit independent raters for C2–C4, and conduct controlled gaming experiments. A production-oriented systems evaluation would additionally require multiple hosts, richer certificate governance, more peers and orderers, privacy controls, load measurements, failures, backup and recovery, and an externally durable artifact store. These extensions would broaden external validity; they are not required to change the completed baseline claim.

7.2.7 FINAL CONCLUSION

This thesis contributes a reproducible path from repository history to auditable ledger evidence. It constructs and validates a multi-project dataset, compares candidate indicators under explicit criteria, identifies and discloses a material release-window ancestry limitation, selects M4 without changing the frozen evidence, and implements the selected signal through deterministic off-chain verification and Hyperledger Fabric chaincode.

Relative to the broad metric and assessment literature and the full DESTINO proposal, this contribution is intentionally focused. It complements the earlier general perspective by operationalizing one selected repository signal as an auditable evidence record; it does not claim to provide a universal developer-quality measure, a complete reputation model, or the complete DESTINO ecosystem.

The final conclusion is deliberately bounded. Under the declared evaluation framework and ancestry-only eligibility gate, M4 developer churn ownership share is a suitable signal for the baseline DESTINO-aligned PoC. The implemented mechanism reproduces the selected fixture from Git, validates exact arithmetic and evidence commitments, and commits the resulting record through authenticated two-organization endorsement with independently confirmed channel state. This establishes a coherent, replayable, and testable PoC. It does not establish that M4 universally measures developer quality, that gaming resistance is empirically proven, or that the educational Fabric topology is production-ready. Within those limits, the research questions are answered and

the baseline thesis objective is achieved.

References

- [1] Centre for Applied Software Engineering (CASE) and University of Padua (UNIPD). *DESTINO: Decentralized Ecosystems for Software Trustworthy INtegratiOn*. TrustChain Third Open Call for Proposals. Undated project proposal supplied as the thesis context.
- [2] Weichao Gao, William G. Hatcher, and Wei Yu. “A Survey of Blockchain: Techniques, Applications, and Challenges”. In: *Proceedings of the 27th International Conference on Computer Communication and Networks (ICCCN)*. 2018, pp. 1–11. DOI: 10.1109/ICCCN.2018.8487348.
- [3] Hyperledger Fabric Project. *Deploying a Smart Contract to a Channel*. Hyperledger Fabric Documentation, https://hyperledger-fabric.readthedocs.io/en/latest/deploy_chaincode.html. Accessed 2026-07-26. 2026.
- [4] Hyperledger Fabric Project. *Hyperledger Fabric v2.5.15 Release Notes*. GitHub release, <https://github.com/hyperledger/fabric/releases/tag/v2.5.15>. Released 2026-02-23; accessed 2026-07-26. 2026.
- [5] Hyperledger Fabric Project. *Using the Fabric Test Network*. Hyperledger Fabric Documentation, https://hyperledger-fabric.readthedocs.io/en/latest/test_network.html. Accessed 2026-07-26. 2026.
- [6] Gerta Kapllani et al. “An Empirical Analysis of the Maintainability Evolution of Open Source Systems”. In: *Open Source Systems*. Springer, 2020, pp. 78–86. DOI: 10.1007/978-3-030-47240-5_8.
- [7] Ilya Khomyakov and Alberto Sillitti. “A Novel Approach for Collecting and Sharing Software Metrics Data”. In: *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*. 2019, pp. 2164–2167. DOI: 10.1145/3297280.3297625.

REFERENCES

- [8] Eleni Koutrouli and Aphrodite Tsalgatidou. “Reputation Systems Evaluation Survey”. In: *ACM Computing Surveys* 48.3 (2015), 35:1–35:28. DOI: 10.1145/2835373.
- [9] Balraj Kumar. “A Survey of Key Factors Affecting Software Maintainability”. In: *2012 International Conference on Computing Sciences*. 2012. DOI: 10.1109/ICCS.2012.5.
- [10] Xiaozhou Li et al. “Exploring Factors and Metrics to Select Open Source Software Components for Integration: An Empirical Study”. In: *Journal of Systems and Software* 188 (2022), p. 111255. DOI: 10.1016/j.jss.2022.111255.
- [11] Tomas Remencius, Alberto Sillitti, and Giancarlo Succi. “Assessment of Software Developed by a Third-Party: A Case Study and Comparison”. In: *Information Sciences* 328 (2016), pp. 237–249. DOI: 10.1016/j.ins.2015.08.028.
- [12] Andreas Schlosser, Marco Voss, and Lars Brueckner. *Comparing and Evaluating Metrics for Reputation Systems by Simulation*. Proceedings of the Workshop on Reputation in Agent Societies (RAS 2004). Local PDF copy in the “Papers to read” collection. 2004.
- [13] Sharifah Mashita Syed-Mohamad et al. “Measuring Software Maintainability: An Analysis of Evolving Metrics across Development Paradigms”. In: *Journal of Advanced Research in Applied Sciences and Engineering Technology* 63.2 (2026), pp. 181–195. DOI: 10.37934/araset.63.2.181195.
- [14] Masoud Jamshidiyan Tehrani. *Assessing Vulnerability in Smart Contracts: The Role of Code Complexity Metrics in Security Analysis*. arXiv preprint. Version 4; revised 2026-01-23. 2024. DOI: 10.48550/arXiv.2411.17343. arXiv: 2411.17343. URL: <https://arxiv.org/abs/2411.17343>.
- [15] Aline Lopes Timóteo et al. *Software Metrics: A Survey*. Survey manuscript. Local PDF copy in the “Papers to read” collection. 2008.
- [16] Roberto Tonelli et al. “Smart Contracts Software Metrics: A First Study”. In: *PLOS ONE* 18.4 (2023), e0281043. DOI: 10.1371/journal.pone.0281043.